Set 1: Short Notes & Definitions
A Scholarly Exposition

Data Marts
Definition and Taxonomy: A data mart constitutes a subset of a data warehouse, architecturally designed to serve the analytical requirements of a specific business unit, department, or user community (e.g., marketing, finance). It represents a focused, denormalized repository of aggregated and historical data optimized for query performance and domain-specific decision support.

Theoretical Context: Debates center on two implementation strategies: dependent data marts (top-down, architecturally consistent with a centralized enterprise data warehouse) versus independent data marts (bottom-up, potentially creating "silos" and integration challenges). The dependent approach aligns with Kimball's conformed dimensions paradigm, while independent marts risk creating inconsistent, non-integrated data islands.

Significance: Data marts reduce query complexity, improve response times, and provide subject-oriented views, though they may introduce redundancy and governance challenges if not properly architected.

Association Rule Generation
Definition: Association rule generation is an unsupervised data mining technique that discovers interesting correlations, frequent patterns, associations, or causal structures among sets of items in transaction databases. Formally, it identifies implications of the form *$X \rightarrow Y$*, where *X* and *Y* are disjoint itemsets.

Theoretical Foundations: The problem is defined by two key metrics: support (frequency of *$X \cup Y$* in the dataset) and confidence (conditional probability $P(Y|X)$). The Agrawal, Imieliński, and Swami (1993) framework established the formal basis for mining such rules, introducing the concept of *frequent itemsets* as antecedents to rule generation.

Applications: Market basket analysis, cross-selling strategies, recommendation systems, and bioinformatics (e.g., gene expression analysis).

Apriori Algorithm
Definition: The Apriori algorithm is a seminal, level-wise, breadth-first search algorithm for mining frequent itemsets and generating association rules. It operates on the fundamental *apriori property*: all nonempty subsets of a frequent itemset must also be frequent.

Algorithmic Mechanics: The procedure involves iterative steps: (1) generate candidate *k*-itemsets from frequent *(k-1)*-itemsets; (2) prune candidates violating the apriori property; (3) scan the database to count support; (4) retain itemsets meeting minimum support threshold. This generate-and-test paradigm, while intuitive, suffers from computational overhead due to multiple database scans.

Complexity Analysis: Time complexity is $O(2^n)$ in worst-case scenarios; I/O costs dominate for large datasets. Subsequent innovations (FP-Growth, Eclat) address these limitations through prefix-tree structures and depth-first search strategies.

Clustering and its Methods

Definition: Clustering is an unsupervised pattern recognition technique that partitions a dataset into homogenous groups (clusters) such that intra-cluster similarity is maximized while inter-cluster similarity is minimized. It constitutes exploratory data analysis without pre-labeled classes.

Taxonomy of Methods:

1. Partitioning Methods: *k-means*, *k-medoids* (PAM) – iterative optimization with spherical cluster assumptions.
2. Hierarchical Methods: Agglomerative (bottom-up merging) and divisive (top-down splitting) – dendrogram representation.
3. Density-Based Methods: DBSCAN, OPTICS – identify arbitrarily shaped clusters based on density connectivity.
4. Grid-Based Methods: STING, CLIQUE – quantize space into grid cells.
5. Model-Based Methods: Gaussian Mixture Models, conceptual clustering – assume underlying statistical distributions.

Theoretical Debates: The "clustering validation" problem remains unresolved; no universally optimal algorithm exists. The choice depends on data characteristics, cluster shape assumptions, and computational constraints.

Bayes' Theorem
Definition: Bayes' Theorem provides a mathematical framework for updating prior beliefs (hypotheses) with observed evidence to derive posterior probabilities. Formally:

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

where *$P(H|E)$* is posterior probability, *$P(E|H)$* is likelihood, *$P(H)$* is prior probability, and *$P(E)$* is marginal probability of evidence.

Epistemological Significance: The theorem underpins Bayesian inference, representing a subjective interpretation of probability. It distinguishes between prior knowledge and posterior knowledge synthesis, forming the basis for Bayesian networks, Naive Bayes classifiers, and probabilistic graphical models.

Historical Context: Attributed to Reverend Thomas Bayes (1763), later refined by Pierre-Simon Laplace. It contrasts with frequentist statistics by treating parameters as random variables rather than fixed constants.

OLTP (Online Transaction Processing)
Definition: OLTP systems are operational database architectures designed to manage real-time, atomic transaction-oriented tasks characterized by high concurrency, rapid response times, and ACID (Atomicity, Consistency, Isolation, Durability) compliance. They support day-to-day business operations through INSERT, UPDATE, and DELETE operations.

Architectural Characteristics: Normalized schemas (typically 3NF) minimize redundancy and ensure data integrity. Examples include ERP systems, banking platforms, and e-commerce order processing.

Distinction from OLAP: OLTP contrasts sharply with Online Analytical Processing (OLAP) systems, which are optimized for complex read-heavy queries and historical analysis. The "OLTP vs. OLAP" dichotomy represents a fundamental architectural separation in modern data ecosystems, bridged by ETL/ELT processes.

Performance Metrics: Measured in transactions per second (TPS), with sub-second latency requirements.

Web Structure Mining
Definition: Web structure mining is a subdiscipline of web mining that analyzes the hyperlink topology of the World Wide Web to discover knowledge about relationships between web pages. It treats the web as a directed graph where nodes represent pages and edges represent hyperlinks.

Theoretical Framework: Building on citation analysis (Garfield's bibliographic coupling), it leverages graph-theoretic metrics (centrality, PageRank, HITS algorithm) to identify authoritative and hub pages. The underlying assumption: hyperlinks constitute implicit endorsements of content quality.

Applications: Search engine ranking, web community detection, website structure optimization, and identification of spam link farms.

Cloud Data Warehousing
Definition: Cloud data warehousing represents the deployment of data warehouse architectures on distributed, virtualized, elastic cloud infrastructure (IaaS/PaaS). It decouples compute and storage resources, enabling on-demand scalability and consumption-based pricing models.

Architectural Paradigms:
- Shared-disk: Centralized storage (e.g., Amazon Redshift)
- Shared-nothing: MPP architectures with distributed storage (e.g., Snowflake, Google BigQuery)
- Serverless: Fully managed, auto-scaling query engines (e.g., Athena)

Theoretical Implications: Challenges traditional capital expenditure (CapEx) models, introduces "data gravity" considerations, and raises sovereignty/privacy concerns. The "lift-and-shift" vs. "cloud-native" design debate reflects fundamental architectural tensions between legacy system migration and greenfield optimization.

Data Granularity
Definition: Data granularity refers to the level of detail or atomicity preserved in a dataset, representing the finest resolution at which data is captured, stored, and analyzed. It exists on a continuum from coarse-grained (highly aggregated) to fine-grained (transaction-level detail).

Trade-off Analysis: The granularity trade-off is fundamental: finer granularity supports more flexible analysis and drill-down capabilities but increases storage costs, query complexity, and processing latency. Coarser granularity improves performance but sacrifices analytical depth.

Kimball's Perspective: The dimensional modeling literature advocates for "the lowest possible granularity" as a design principle to future-proof analytical capabilities while using aggregation strategies (e.g., materialized views) for performance optimization.

Data Transformation (Data Preprocessing Context)
Definition: Data transformation is the preprocessing stage that converts raw data into an appropriate, normalized format suitable for mining and analysis through application of mathematical functions, mapping operations, or algorithmic modifications.

Taxonomy of Transformations:
- Smoothing: Noise reduction (binning, regression, clustering)
- Aggregation: Summarization to higher conceptual levels
- Generalization: Concept hierarchy climbing

- Normalization: Feature scaling (min-max, z-score, decimal scaling)
- Attribute Construction: Deriving new attributes from existing ones
- Discretization: Converting continuous to categorical variables

Theoretical Rationale: Addresses violations of algorithmic assumptions (e.g., distance sensitivity in k-means), mitigates scale dominance, and improves computational efficiency. The transformation pipeline critically influences model validity—a source of "garbage in, garbage out" failures.

Data Warehouse Automation
Definition: Data warehouse automation (DWA) refers to the application of metadata-driven frameworks, design patterns, and tooling to automatically generate ETL/ELT code, schema definitions, documentation, and deployment artifacts, thereby reducing manual development effort.

Methodological Approach: DWA platforms (e.g., WhereScape, Dimodelo) embed data modeling best practices and generate platform-specific code from logical models. It implements agile data warehousing principles through rapid prototyping and iterative refinement.

Academic Debate: Proponents argue it accelerates time-to-value and enforces standardization. Critics caution against "black box" generation that may obscure optimization opportunities and create technical debt if underlying metadata is poorly governed.

Data Lake and its Architecture
Definition: A data lake is a centralized repository that stores massive volumes of raw, unprocessed data in its native format (structured, semi-structured, and unstructured) until required for analysis. It embodies a "store first, schema-on-read" philosophy.

Architectural Layers:
1. Ingestion Layer: Batch and streaming data acquisition
2. Storage Layer: Distributed file systems (HDFS, S3, ADLS) with object storage
3. Processing Layer: Compute engines (Spark, Flink) for transformation
4. Governance Layer: Metadata catalog (Data Catalog), access control, lineage tracking
5. Consumption Layer: Analytics tools, BI platforms, ML frameworks

Theoretical Tension: The "data swamp" or "data lake anti-pattern" emerges when governance is neglected, creating an unusable, undocumented data dump. This highlights the dialectic between flexibility and control.

ELT vs. ETL
Conceptual Distinction: ELT (Extract, Load, Transform) and ETL (Extract, Transform, Load) represent two paradigms for data integration, differing fundamentally in *where* transformation occurs relative to the target system.

ETL (Traditional):
- Process: Extract → Transform (in staging area) → Load into warehouse
- Rationale: Transformation occurs on dedicated middleware, reducing warehouse compute load. Optimized for on-premises, structured data, and pre-defined schemas.
- Limitations: Rigid, schema-on-write, scalability constraints.

ELT (Modern):
- Process: Extract → Load into lake/warehouse → Transform (using target compute)

- Rationale: Leverages elastic cloud compute for transformations; supports raw data ingestion and schema-on-read. Enables agile analytics and data science exploration.
- Implications: Requires robust governance; transformation costs shift to storage platform.

Theoretical Shift: ELT reflects the "democratization of data processing" and the rise of data lakes, challenging traditional data warehouse hub-and-spoke architectures.

Applications of Data Mining
Domain-Specific Applications:

1. Retail & Marketing: Customer segmentation, churn prediction, market basket analysis, recommendation engines
2. Finance: Credit scoring, fraud detection, risk modeling, algorithmic trading
3. Healthcare: Disease diagnosis (SVMs, ANNs), drug discovery, patient outcome prediction
4. Telecommunications: Network intrusion detection, fault prediction, subscriber profiling
5. Manufacturing: Quality control, predictive maintenance, supply chain optimization
6. Scientific Research: Bioinformatics (genome sequencing), climate modeling, astronomical data analysis
7. Web & Social Media: Sentiment analysis, trending topic detection, social network analysis

Societal Implications: Raises ethical concerns regarding privacy (GDPR), algorithmic bias, and surveillance capitalism. The "dual-use" problem highlights potential misuse for discrimination and manipulation.

OLAP Data Cube Operations
Definition: OLAP cube operations are multidimensional analytical primitives that enable intuitive navigation and aggregation of data across dimensions, implementing the slice-and-dice metaphor.

Fundamental Operations:
- Slice: Selecting a single dimension value, creating a sub-cube (e.g., sales for *Year=2023*)
- Dice: Selecting multiple dimension ranges, creating a smaller hyper-rectangle
- Roll-up: Aggregating data upward through a dimension hierarchy (e.g., day → month → quarter)
- Drill-down: Disaggregating to finer granularity, exposing more detail
- Pivot (Rotate): Reorienting the dimensional axes for alternative perspective
- Drill-across: Navigating between related cubes sharing conformed dimensions

Mathematical Formalism: Operations correspond to relational algebra projections, selections, and aggregations, optimized through pre-computed materialized views.

Aggregate Fact Table and Derived Dimensional Tables
Definition: An aggregate fact table is a pre-computed summary table storing aggregated metrics (e.g., SUM, COUNT) at higher granularity levels than the base atomic fact table, created to accelerate query performance.

Derived Dimensional Tables: Created by denormalizing or summarizing base dimension tables, often incorporating aggregated attributes (e.g., "CustomerLifetimeValue") or hierarchical rollups.

Design Rationale: Implements Kimball's "aggregate navigation" principle. The trade-off between space vs. time: aggregates consume storage but reduce query latency by orders of magnitude. Requires careful maintenance to ensure synchronization with base tables (slowly changing dimensions challenge).

Academic Critique: Over-reliance on aggregates may indicate poor base table design; modern columnar storage and MPP query engines reduce necessity, shifting toward "just-in-time" aggregation.

Data Swamp
Definition: A data swamp is the pathological degeneration of a data lake into an unusable, undocumented, and ungoverned repository where data quality, lineage, and metadata are compromised, rendering analytical efforts inefficient or impossible.

Etiology: Results from inadequate governance, absent metadata management, indiscriminate data ingestion without curation, and lack of stewardship. The "store everything" philosophy without "manage everything" rigor.

Diagnostic Criteria:
- Metadata catalog absent or outdated
- Data lineage untraceable
- Duplicate, orphaned, and obsolete datasets proliferate
- Security and compliance violations endemic
- Analyst time predominantly spent on data discovery (>80%)

Prescription: Implement robust data governance frameworks, automated metadata harvesting, data quality profiling, lifecycle policies, and stewardship roles. Emphasizes that *technical architecture alone cannot solve organizational dysfunction*.

Data Preprocessing Stages
The Data Preparation Pipeline:
1. Data Cleaning: Handling missing values (imputation, deletion), noise smoothing, outlier detection
2. Data Integration: Merging heterogeneous sources; resolving schema and semantic conflicts
3. Data Reduction: Dimensionality reduction (PCA, feature selection), numerosity reduction (sampling, aggregation)
4. Data Transformation: Normalization, discretization, concept hierarchy generation
5. Data Quality Assessment: Profiling, validation against business rules

Theoretical Justification: Preprocessing addresses the "data quality dilemma"—real-world data is inherently dirty, incomplete, and inconsistent. GIGO (Garbage In, Garbage Out) principle mandates that model performance is fundamentally bounded by input quality.

Empirical Studies: Consensus suggests 60-80% of data science project effort is consumed by preprocessing, highlighting its criticality despite being undervalued academically.

Agglomerative Approach of Hierarchical Method
Definition: Agglomerative hierarchical clustering is a bottom-up, greedy algorithm that initiates with each data point as an individual cluster and iteratively merges the closest pair of clusters until a single cluster remains or a termination condition is met.

Algorithmic Procedure:
1. Compute proximity matrix between all pairs
2. Merge most similar clusters (based on linkage criterion)
3. Update proximity matrix
4. Repeat until convergence

Linkage Criteria (Dendrogram Branching Strategy):
- Single Linkage: Minimum distance (produces "chaining" effect)
- Complete Linkage: Maximum distance (favors compact clusters)
- Average Linkage: Mean distance (balanced approach)
- Ward's Method: Minimum variance increase (statistically principled)

Computational Complexity: $O(n^3)$ time, $O(n^2)$ space—prohibitive for large datasets. Dendrogram visualization provides hierarchical structure interpretation but requires subjective cutting depth determination.

Metadata and Data Warehousing
Definition: Metadata is "data about data"—structured information describing data assets' structure, meaning, origin, usage, and governance characteristics. In warehousing, it forms the *lingua franca* enabling comprehension across technical and business stakeholders.

Metadata Categories:
- Technical Metadata: Schema definitions, data types, ETL mappings, lineage, transformation logic
- Business Metadata: Business definitions, KPI formulas, stewardship, glossary terms
- Operational Metadata: Load timestamps, job execution statistics, error logs

Theoretical Significance: Metadata implements the "active data warehouse" concept, enabling automated impact analysis, self-service BI, and regulatory compliance (GDPR's "right to be forgotten" requires lineage metadata). Poor metadata governance is cited as the primary cause of warehouse failure in longitudinal studies.

Rule-Based Classification
Definition: Rule-based classification uses a collection of IF-THEN propositional rules (*R*) of the form: IF *condition* THEN *class*, where the condition is a conjunction of attribute-value tests.

Rule Generation Strategies:
- Direct Method: Sequential covering algorithms (e.g., RIPPER, FOIL) learn rules directly from data
- Indirect Method: Extract rules from decision trees (C4.5rules) or other models

Evaluation Metrics: Rule quality is assessed by coverage (fraction of records satisfying antecedent) and accuracy (fraction of covered records correctly classified). The rule set quality requires handling overlapping rules through voting, ordering, or conflict resolution.

Advantages: Interpretable, intuitive for domain experts, no black-box nature. Limitations: Fragile with high-dimensional data; struggles with continuous attributes; may produce overly complex rule sets.

Naive-Bayes Classifier
Definition: The Naive Bayes classifier is a probabilistic, generative model applying Bayes' theorem with the "naive" assumption of conditional independence between features given the class label.

Mathematical Formulation:

$$P(C|F_1,...,F_n) \propto P(C) \prod_{i=1}^{n} P(F_i|C)$$

where class *C* is predicted by maximizing posterior probability.

Variants:
- Gaussian NB: Continuous features assumed normally distributed
- Multinomial NB: Discrete counts (text classification)
- Bernoulli NB: Binary features

Theoretical Puzzle: Despite violating independence assumptions in most real-world domains, Naive Bayes demonstrates remarkable empirical effectiveness, especially in high-dimensional spaces (text classification). This "naive Bayes paradox" is attributed to classification being robust to likelihood estimation errors when posterior probabilities remain correctly ordered.

Computational Advantage: $O(n)$ training time—single pass through data to estimate parameters. Optimal for baseline models and real-time classification.

Page Rank (Definition and Context)
Definition: PageRank is a link analysis algorithm that assigns a numerical weight to each web page, measuring its relative importance within the web graph. It models a random surfer who follows links with probability *d* (damping factor) and teleports randomly with probability *1-d*.

Mathematical Model: Solves the eigenvector problem:

$$PR(p_i) = \frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)}$$

where *$M(p_i)$* are pages linking to *p_i*, and *$L(p_j)$* is the number of outbound links from *p_j*.

Theoretical Context: Developed by Brin and Page (1998) at Stanford, forming Google's core ranking mechanism. Represents a stationary distribution of a Markov chain over the web graph. The damping factor addresses sink nodes and ensures ergodicity.

Extensions: Topic-sensitive PageRank, Personalized PageRank, TrustRank (combating spam). Limitations: Vulnerable to link farms; ignores content relevance; computational expense for dynamic graphs.

Vision Page Segmentation
Definition: Vision-based page segmentation (VIPS) is a web content analysis technique that partitions a web page into semantically coherent blocks by analyzing visual layout cues rendered in the browser, rather than relying solely on HTML DOM structure.

Methodology:
1. Render page to extract visual features: font size, color, spatial coordinates, separators
2. Construct vision-based content structure (Visual Block Tree)
3. Detect implicit separators (horizontal/vertical gaps, lines)
4. Assign degree of coherence to each block using vision-based features

Theoretical Advancement: Addresses the limitation that HTML structure poorly represents semantic organization due to CSS, tables-for-layout, and dynamic content. VIPS achieves higher segmentation accuracy than DOM-based methods for information retrieval and wrapper induction.

Applications: Web scraping, mobile adaptation, search result diversification, block-level indexing.

Page Layout Analysis
Definition: Page layout analysis is the computational process of identifying and categorizing regions of interest in document images (web pages, scanned documents) to understand spatial arrangement and reading order.

Two Sub-problems:
- Physical Layout: Geometric segmentation (text blocks, images, tables, lines) using computer vision (connected components, Voronoi diagrams)
- Logical Layout: Semantic role labeling (title, abstract, caption, footnote) using machine learning

Theoretical Framework: Combines bottom-up (pixel-to-region) and top-down (template matching) strategies. The Manhattan layout assumption (rectangular, non-overlapping blocks) simplifies many algorithms but fails for complex designs.

Challenges: Multi-column layouts, overlapping elements, reading order ambiguity (critical for accessibility). Modern approaches use deep learning (CNNs, GCNs) for end-to-end segmentation.

Concept of Noisy Data Cleaning
Definition: Noisy data cleaning is the preprocessing activity identifying and rectifying random errors, distortions, or artifacts in data that obscure underlying patterns, thereby degrading model performance and analytical validity.

Noise Typology:
- Measurement Noise: Instrument inaccuracies, sensor drift
- Human Error: Data entry mistakes, transcription errors
- Systemic Artifacts: Parsing errors, encoding issues, transmission corruption

Cleaning Strategies:
- Smoothing: Binning, moving averages, LOWESS regression
- Clustering-Based: Detect outliers as noise points
- Regression-Based: Fit models and retain residuals within thresholds
- Human-in-the-Loop: Expert validation for ambiguous cases

Philosophical Tension: The distinction between noise and legitimate outliers (anomalies) is ambiguous. Aggressive cleaning may remove informative rare events. The robust statistics literature advocates methods insensitive to noise rather than wholesale removal.

Operational Data Store (ODS)
Definition: An ODS is an integrated, subject-oriented, volatile, current-valued data store containing detailed, near real-time operational data extracted from source systems, designed for tactical decision-making and operational reporting.

Characteristics:
- Volatile: Data is overwritten, not historically preserved
- Near Real-Time: Latency typically minutes to hours
- Enterprise-Wide Integration: Consolidated view across operational systems
- Detailed Granularity: Atomic transaction-level data

Architectural Positioning: Positioned between source OLTP systems and the strategic data warehouse. Serves as a "day-to-day" reporting layer without impacting production OLTP performance.

Distinction from DWH: ODS lacks historical depth and non-volatility (Inmon's defining criteria). Complements rather than replaces the warehouse.

Enterprise Data Warehouse (EDW)
Definition: An EDW is a centralized, integrated, non-volatile, time-variant repository of historical data representing a single, consistent version of truth for the entire enterprise, supporting strategic decision-making.

Inmon's Formal Criteria (CIF):
- Subject-Oriented: Organized by business domains
- Integrated: Standardized semantics across sources
- Non-Volatile: Read-only, appended, not overwritten
- Time-Variant: Historical perspective (5-10+ years)

Architectural Philosophy: Implements a top-down, normalized approach (3NF) for enterprise-wide consistency. Contrasts with Kimball's bottom-up dimensional modeling (star schemas in departmental data marts).

Strategic Value: Enables cross-functional analytics, regulatory compliance, and longitudinal trend analysis. High implementation complexity and cost; requires strong governance.

Dependent vs. Independent Data Marts
Taxonomical Distinction:

Dependent Data Marts:
- Origin: Sourced from a centralized EDW or data lake
- Architecture: Top-down; conforms to enterprise data model
- Advantages: Consistent definitions, single source of truth, reduced redundancy
- Trade-offs: Slower to deploy; EDW bottleneck

Independent Data Marts:
- Origin: Directly sourced from operational systems
- Architecture: Bottom-up; department-specific
- Advantages: Rapid deployment; responsive to local needs; lower initial cost
- Trade-offs: Inconsistent definitions ("silos"); integration challenges; analytic disparity

Theoretical Debate: The "data mart consolidation" problem describes the costly refactoring required when independent marts proliferate. Modern "data mesh" architecture attempts to resolve this through domain-oriented decentralized ownership with federated governance.

Data Puddle, Data Pond, and Data Ocean (Stages of Data Lake)
Evolutionary Metaphor:

Data Puddle (Stage 1):
- Definition: A single-purpose, isolated data store serving one team or project
- Characteristics: Lacks governance, minimal structure, ad-hoc creation
- Risk: Precursor to silos; may metastasize into uncontrolled proliferation

Data Pond (Stage 2):
- Definition: Evolved puddle with multiple contributors, basic metadata, and some governance

- Characteristics: Department-level scope; emerging standards; semi-structured
- Challenge: Integration gaps between ponds; inconsistent semantics

Data Ocean (Stage 3):
- Definition: Fully integrated, enterprise-scale data lake with comprehensive governance, metadata, and cross-domain linkages
- Characteristics: Self-service analytics; data as a product; federated governance
- Ideal State: Realizes the data lake vision—democratized access with control

Critical Perspective: The metaphor reveals that "data lake" is not a static architecture but a maturity journey. Most organizations remain stuck at "puddle" stage, explaining the high failure rates reported in industry surveys.

End of Set 1.

Comprehensive Academic Responses: Data Warehousing & Mining

I'll address each question with the depth, rigor, and pedagogical structure expected at the graduate level. Each response integrates theoretical foundations with practical applications.

---

1. Data Warehouse: Definition and Characteristics

Definition:
A Data Warehouse is a subject-oriented, integrated, non-volatile, and time-variant collection of data designed to support management's decision-making process (Inmon, 1992). It represents a centralized repository that aggregates data from multiple heterogeneous sources, transforming it into a consistent format optimized for analytical querying rather than transactional processing.

The Four Defining Characteristics:

a) Subject-Oriented: Data is organized around major business subjects (e.g., customers, products, sales) rather than application processes. This facilitates analytical queries by consolidating all relevant data about a subject. For instance, a "customer" subject area would integrate data from sales, marketing, and support systems into a unified view.

b) Integrated: Data from disparate operational systems (ERP, CRM, legacy systems) is consolidated with consistent naming conventions, formats, and semantics. Integration resolves conflicts: for example, date fields standardized to YYYY-MM-DD, currency values converted to USD, and customer IDs unified across platforms. This creates a "single version of truth."

c) Non-Volatile: Once entered, data is not updated or deleted—only appended. This immutability preserves historical accuracy and enables trend analysis. Operational systems overwrite records (e.g., changing a customer's address), while warehouses maintain both old and new addresses with effective dates.

d) Time-Variant: Data is tagged with time dimensions, maintaining historical snapshots typically spanning 5-10 years. This enables temporal analysis like "How have customer purchasing patterns evolved quarterly?" Unlike OLTP systems that show current state only, warehouses track evolutionary changes.

---

2. Data Cleaning: Noisy Data Handling

Data Cleaning is the preprocessing phase that identifies and rectifies corrupt, inaccurate, or irrelevant records to improve data quality for reliable analysis.

Noisy Data Cleaning:
Noisy data contains random errors or outliers—values that deviate significantly from expected patterns due to measurement faults, data entry errors, or system glitches.

Examples and Techniques:

* Sensor Malfunction: A temperature sensor reading 500°F in a refrigerated truck (outlier detection using z-score > 3σ).
* Data Entry Error: Age recorded as "250" years (range validation).
* Outlier Treatment Methods:
  - Binning: Smooth sorted values by neighborhood averaging
  - Regression: Fit data to mathematical functions to smooth values
  - Clustering: Detect and remove values outside cluster boundaries (e.g., DBSCAN)
  - Human Inspection: Manual review of suspicious records flagged by automated tools

---

3. Decision Trees in Classification

Decision Tree is a hierarchical, flowchart-like structure where internal nodes represent feature tests, branches denote test outcomes, and leaf nodes represent class labels. It partitions data recursively based on attribute selection metrics.

Utility in Classification:
Decision Trees provide interpretable classification models that humans can comprehend, unlike black-box algorithms. They work by:

1. Attribute Selection: Using information gain (ID3), gain ratio (C4.5), or Gini impurity (CART) to identify the most discriminative feature at each node
2. Recursive Partitioning: Splitting data subsets until purity thresholds are met
3. Pruning: Reducing overfitting by removing branches that add minimal predictive value

Example: Classifying loan approval using income > $50k, credit score > 700, and debt-to-income ratio. The tree reveals decision logic transparently for regulatory compliance.

---

4. Text Mining: Concepts and Techniques

Text Mining is the computational process of extracting high-quality information from unstructured textual data through pattern discovery, statistical analysis, and machine learning.

Applications: Sentiment analysis, document clustering, information retrieval, spam detection, legal e-discovery, biomedical literature mining, and customer feedback analysis.

Two Key Techniques:

a) Named Entity Recognition (NER):
Identifies and classifies named entities (persons, organizations, locations) into predefined categories using sequence labeling models like Conditional Random Fields or BERT.

Example: In "Apple Inc. announced new iPhones in Cupertino," NER tags "Apple Inc." as ORGANIZATION and "Cupertino" as LOCATION, enabling extraction of business intelligence.

b) Topic Modeling (LDA):
Discovers abstract topics that occur in document collections using generative probabilistic models. Each document is modeled as a mixture of topics, where each topic is a probability distribution over words.

Example: Analyzing 10,000 research papers to automatically identify emerging research themes (e.g., "deep learning," "quantum computing") without predefined categories.

---

5. Data Lake: Creation Process

Data Lake is a centralized repository that stores massive volumes of raw, unprocessed data in native format (structured, semi-structured, unstructured) until needed for analysis, providing schema-on-read flexibility.

Step-by-Step Creation:

1. Architectural Design: Define storage tiers (hot/warm/cold), security zones, and governance policies. Choose between cloud (AWS S3, Azure ADLS) or on-premise (Hadoop HDFS) infrastructure.

2. Data Ingestion: Implement robust ingestion pipelines using tools like Apache NiFi, Flume, or Kafka. Support batch, micro-batch, and streaming ingestion with metadata capture.

3. Storage Organization: Apply hierarchical directory structures (e.g., `/raw/{source}/{date}/`) and file formats (Parquet, ORC for columnar storage; Avro for schema evolution).

4. Metadata Cataloging: Deploy a data catalog (AWS Glue, Apache Atlas) to maintain technical, business, and operational metadata, preventing the lake from becoming a "data swamp."

5. Access Control & Security: Implement role-based access control (RBAC), encryption at rest/transit, and data masking for PII compliance with GDPR/CCPA.

6. Governance Framework: Establish data quality monitoring, lifecycle policies (retention/archival), and stewardship roles to ensure discoverability and trustworthiness.

7. Analytics Enablement: Provide query engines (Presto, Spark SQL) and machine learning platforms for data scientists to explore and derive value.

---

6. Metadata in Data Warehousing

Metadata is "data about data"—structured information that describes, explains, or makes it easier to retrieve, use, or manage data resources.

Contents Include:

- Technical Metadata: Data structures, schemas, field types, lengths, transformation rules, data lineage, ETL job schedules
- Business Metadata: Business definitions, KPI formulas, ownership, stewardship contacts, data sensitivity classifications
- Operational Metadata: Load timestamps, record counts, error logs, refresh frequencies, usage statistics

Importance in Data Warehousing:
Metadata transforms a warehouse from a "black box" into a discoverable, governable asset. It enables:
- Impact Analysis: Tracing how changes to source systems affect downstream reports
- Data Lineage: Understanding data origins and transformations for audit compliance
- Self-Service BI: Empowering business users to find trusted datasets autonomously
- Query Optimization: Assisting optimizers with statistics for execution planning

Types:
1. Descriptive Metadata: Discovery and identification (titles, authors, keywords)
2. Structural Metadata: Formal relationships (schemas, indexes, partitions)
3. Administrative Metadata: Technical management (rights, permissions, provenance)

---

7. ETL Components and Performance Optimization

ETL (Extract, Transform, Load) is the foundational process for populating data warehouses.

Three Components:

a) Extract: Retrieve data from heterogeneous sources (databases, APIs, flat files) using full extraction, incremental extraction (CDC), or threshold-based techniques. Challenges include minimizing source system impact and handling source schema changes.

b) Transform: Apply cleansing, validation, aggregation, enrichment, and business rule implementations. Operations include data type conversion, surrogate key generation, slowly changing dimension (SCD) handling, and denormalization.

c) Load: Populate warehouse targets using bulk loading utilities (SQL*Loader, bcp), transaction management, and index maintenance strategies. Includes initial loads and incremental refresh cycles.

Performance Improvement Strategies:

- Parallel Processing: Partition data horizontally/vertically; use multi-threaded ETL engines
- Pushdown Optimization: Execute transformations at source or warehouse DBMS using SQL pushdown
- Staging Area: Use persistent staging tables to avoid re-extraction on failure
- Change Data Capture (CDC): Process only modified records using timestamps, triggers, or log-based CDC
- Bulk Operations: Disable constraints/indexes during load; use bulk insert APIs
- Data Quality Firewall: Validate early in pipeline to prevent propagation of bad data
- In-Memory Processing: Utilize ETL tools with in-memory caches for lookups
- Workload Management: Prioritize critical path jobs; schedule non-critical tasks during off-peak hours

---

8. Association Rule Mining Concepts

Association Rule Mining discovers interesting co-occurrence relationships (affinities) among items in transaction databases, famously applied to market basket analysis.

Core Concepts:

Support Count: Absolute frequency of an itemset in transactions. For rule {Bread}→{Butter}, support count = number of transactions containing both Bread and Butter.

Support (s): Percentage of transactions containing the itemset.
```
s(X→Y) = σ(X∪Y) / N
```
Where σ is support count, N is total transactions.

Frequent Itemset: An itemset whose support ≥ minimum support threshold (minsup). These form the basis for generating strong rules.

Confidence (c): Conditional probability that transactions containing X also contain Y.
```
c(X→Y) = σ(X∪Y) / σ(X)
```

Rule Evaluation Metrics:
- Lift: Measures independence between items. Lift > 1 indicates positive correlation.
- Conviction: Measures rule dependency. Higher values indicate stronger implication.
- Leverage: Difference between observed and expected frequency if items were independent.

Example: In 1000 transactions, {Milk, Bread} appears 300 times (support=30%). If {Milk} appears 500 times, confidence(Milk→Bread)=60%. With lift=1.2, milk and bread are positively correlated, suggesting cross-promotion opportunities.

---

9. Classification: Descriptive vs. Predictive Modeling

Classification maps data instances into predefined categorical classes by learning a decision boundary from labeled training data.

Descriptive Modeling:
Aims to interpret and explain the classification process, providing human-understandable patterns. The model itself is the primary output, revealing feature-class relationships.

Example: Generating a decision tree to explain why loans default (income < $40k AND credit score < 600). Bank regulators require interpretable rules for compliance.

Predictive Modeling:
Focuses on accurate class prediction for unseen data, often sacrificing interpretability for performance. The model acts as a black-box prediction engine.

Example: Using a deep neural network (99.2% accuracy) to predict cancer from MRI scans. While doctors can't interpret individual neuron weights, diagnostic accuracy is paramount.

Trade-off: Descriptive models (logistic regression, decision trees) prioritize transparency; predictive models (ensemble methods, neural networks) prioritize accuracy. Domain requirements dictate the choice.

---

10. Web-Usage Pattern Analysis Techniques

Web Usage Mining extracts behavioral patterns from web log data to optimize site structure, personalization, and marketing.

Techniques:

1. Association Rule Discovery: Identifies page co-occurrence patterns. *Example:* 65% of users who visit `/products/laptop` also visit `/accessories/mouse`, suggesting bundling opportunities.

2. Sequential Pattern Mining: Uncovers navigational sequences over time using algorithms like GSP or PrefixSpan. *Example:* Path analysis reveals 40% follow: Homepage → Search → Product → Cart → Abandon, indicating checkout friction.

3. Clustering: Groups users by behavior using k-means or hierarchical clustering on session features (page views, duration, click depth). Enables persona development: "browsers" vs. "quick buyers."

4. Classification: Predicts user categories (e.g., will purchase) using features like time-on-page, referral source, and device type. Supports real-time personalization.

5. Markov Models: Predicts next-page probabilities based on current state. Used for prefetching content and dynamic navigation recommendations.

---

11. OLAP Operations

OLAP (Online Analytical Processing) enables multidimensional data analysis through specialized operators:

Four Fundamental Operations:

a) Roll-Up (Aggregation): Summarizes data by climbing hierarchy or dimension reduction. *Example:* Rolling monthly sales up to quarterly or yearly totals; removing "Product" dimension to view regional totals only.

b) Drill-Down: Inverse of roll-up—navigates from summarized to detailed data along hierarchies. *Example:* Drilling from yearly sales into quarterly, monthly, and daily granularity for trend analysis.

c) Slice: Creates a sub-cube by selecting one dimension at a specific value. *Example:* Slicing sales cube on `Year=2023` produces a 2D plane of product vs. region for that year only.

d) Dice: Creates a sub-cube by selecting multiple dimensions with specific value ranges, extracting a smaller multi-dimensional block. *Example:* Dicing to analyze sales where `Region IN ('North','South')` AND `Product IN ('Laptop','Tablet')` AND `Quarter BETWEEN 1 AND 2`.

---

12. Data Transformation Steps

Data Transformation converts cleaned data into warehouse-compatible formats suitable for analysis.

Steps:

1. Smoothing: Remove noise using binning, regression, or clustering to highlight patterns.

2. Aggregation: Summarize data to appropriate granularity (e.g., daily sales from transactional records).

3. Generalization: Replace low-level concepts with higher-level abstractions using concept hierarchies (e.g., city → state → region).

4. Normalization: Scale numeric attributes to standard ranges:
  - Min-Max: `[0,1]`
  - Z-score: Mean=0, SD=1
  - Decimal scaling

5. Attribute Construction: Derive new attributes from existing ones (e.g., `Age` from `BirthDate`, `Profit` = `Revenue - Cost`).

6. Discretization: Convert continuous attributes into intervals (e.g., age groups: 0-18, 19-35, 36-55, 55+).

7. Recoding: Transform categorical values (e.g., "Male"/"Female" to 1/0) or handle NULL representations consistently.

8. Key Restructuring: Generate surrogate keys to maintain warehouse independence from source systems.

---

13. Data Mining vs. Data Warehousing

Data Mining is the process of discovering patterns, correlations, anomalies, and insights from large datasets using statistical, machine learning, and pattern recognition techniques. It answers "What hidden knowledge exists?"

Key Differences:

Dimension	Data Warehousing	Data Mining
Purpose	Data integration & storage for reporting	Pattern discovery & predictive modeling
Process	ETL, storage, retrieval	Algorithms, model building, evaluation
Output	Integrated, cleaned datasets for OLAP	Models, patterns, predictions, rules
Orientation	Retrospective (what happened?)	Prospective (what will happen?)
User	Business analysts, managers	Data scientists, statisticians
Query Type	Pre-defined, structured (SQL/MDX)	Ad-hoc, complex algorithms
Data Volume	Structured, aggregated historical data	Raw, detailed data (often from warehouse)

Analogy: The warehouse is the library; data mining is research using that library to discover new knowledge.

---

14. ETL Necessity in Data Warehouses

ETL is indispensable because operational systems are designed for transaction processing, not analytics:

Why ETL is Required:

1. Source Heterogeneity: Enterprises have 50-100+ systems with incompatible schemas, formats (XML, JSON, CSV), and platforms. ETL provides a unified integration layer.

2. Data Quality Assurance: Operational data contains duplicates, inconsistencies, and missing values. ETL implements validation rules (e.g., reject orders with negative quantities) that don't exist in source systems.

3. Historical Integrity: Source systems overwrite history; ETL captures changes using SCD types 1/2/3 to maintain temporal accuracy for trend analysis.

4. Performance Optimization: Warehouse schemas (star/snowflake) are denormalized for query speed, requiring aggregation and transformation that would cripple OLTP systems.

5. Business Logic Consolidation: Centralizes complex calculations (customer lifetime value, risk scores) for consistent reporting across the enterprise.

6. Change Management: Handles source system schema evolution gracefully through metadata-driven mappings, insulating warehouses from upstream disruptions.

Without ETL, a warehouse would be a chaotic "data dump" with no trust, performance, or analytical value.

---

15. Data Mining Issue Categories

Mining Methodology Issues:
- Mining Different Kinds of Knowledge: Databases require diverse algorithms (association, classification, clustering). Single methods are insufficient.
- Interactive Mining: Static mining is limiting; need human-guided, iterative refinement.
- Incorporation of Background Knowledge: Domain constraints (e.g., medical ontologies) should guide mining for semantic correctness.
- Data Mining Query Languages: Lack of standardized syntax (like SQL) for specifying mining tasks.

Performance-Based Issues:
- Efficiency & Scalability: Algorithms must handle terabyte-scale datasets with linear/polynomial complexity.
- Parallel & Distributed Mining: Need algorithms that scale across GPU/CPU clusters (MapReduce, Spark).
- Incremental Mining: Databases evolve; models must update without full retraining on entire dataset.

Diverse Data Types Issues:
- Complex Data: Mining multimedia, spatial, time-series, and graph data requires specialized similarity measures.
- Web Data: Semi-structured HTML/XML with hyperlinks needs hybrid approaches.
- Data Streams: Real-time mining with single-pass algorithms and limited memory (concept drift).
- Privacy & Security: Mining encrypted or anonymized data while preserving utility.

---

16. Handling Missing Values and Noisy Data

Missing Values Strategies:

1. Ignore Tuple: Delete records with missing values (only if <5% of dataset and MCAR - Missing Completely At Random).

2. Manual Imputation: Domain expert fills values (impractical for large datasets).

3. Central Tendency Imputation:
  - Mean for symmetric numeric data
  - Median for skewed numeric data
  - Mode for categorical data

4. Advanced Imputation:
  - Regression: Predict missing values from other attributes
  - K-NN Imputation: Use k nearest neighbors' values
  - EM Algorithm: Iteratively impute using maximum likelihood
  - Multiple Imputation: Generate multiple plausible values to account for uncertainty

5. Distinct Value: Create "Unknown" category for categorical data.

Noisy Data Handling:

1. Binning Methods:
  - Equal-width: Divide range into N intervals; replace values with bin mean/median
  - Equal-frequency: Each bin contains same number of observations (more robust to skew)

2. Regression: Smooth data by fitting to linear/multivariate regression functions.

3. Outlier Analysis:
  - Clustering: Remove values not belonging to any cluster (DBSCAN)
  - Statistical Tests: Discard values beyond ±3σ (assuming normal distribution)
  - IQR Method: Remove values outside [Q1-1.5×IQR, Q3+1.5×IQR]

---

17. K-Means Algorithm

Algorithm Steps:

```
Input: Dataset D = {x_1, x_2, ..., x_n}, number of clusters k
Output: k clusters C = {C_1, C_2, ..., C_k}

1. Initialization: Randomly select k data points as initial centroids μ_1, μ_2, ..., μ_k
2. Repeat until convergence:
  a. Assignment: For each point x_i, assign to cluster C_j where:
     j = argmin_k ||x_i - μ_k||² (Euclidean distance)
  b. Update: For each cluster C_k, recompute centroid:
     μ_k = (1/|C_k|) × Σ_{x∈C_k} x
3. Termination: When centroids stabilize or max iterations reached
```

How It Works:
K-Means minimizes within-cluster sum of squares (WCSS) through iterative optimization. The algorithm:
- Partitions space into Voronoi cells based on nearest centroid
- Optimizes via coordinate descent: alternating between fixing centroids and reassigning points
- Converges to local optimum (not guaranteed global optimum)

Complexity: O(n×k×i×d) where n=points, k=clusters, i=iterations, d=dimensions. Converges in 10-20 iterations typically.

Initialization Sensitivity: Poor initial centroids lead to suboptimal clusters. Use k-means++ for better initialization.

---

18. Text Dimensionality Reduction, Tokenization, Vectorization

Why Reduce Dimensionality?
Text data creates high-dimensional sparse vectors (vocabulary size = 10,000-1,000,000+ features). This causes:
- Curse of Dimensionality: Distance metrics lose meaning; models overfit
- Computational Inefficiency: $O(d^2)$ complexity for many algorithms
- Storage Overhead: Sparse matrices waste memory
- Noise: Many terms are non-discriminative ("the", "a")

Tokenization:
The process of splitting raw text into atomic units (tokens):
- Word Tokenization: `"The cat sat"` → `["The", "cat", "sat"]`
- Subword Tokenization: `"unhappiness"` → `["un", "happiness"]` (handles rare words)
- N-grams: `"the cat sat"` → `["the cat", "cat sat"]` (preserves local context)
- Tools: NLTK, spaCy, Hugging Face tokenizers

Vectorization Approaches:

Bag-of-Words (BoW): Creates term-frequency vectors ignoring order.

```
Corpus: ["cat sat", "cat ran"]
Vocabulary: {cat:0, sat:1, ran:2}
Vectors: [[1,1,0], [1,0,1]]
```

TF-IDF (Term Frequency-Inverse Document Frequency):
Weights terms by rarity to highlight discriminative words:

```
TF(t,d) = (count of t in d) / (words in d)
IDF(t) = log(N / df(t)) where N=total docs, df(t)=docs containing t
TF-IDF(t,d) = TF(t,d) × IDF(t)
```

Example: "Data" appears in all docs → low IDF; "warehouse" appears rarely → high IDF.

---

19. Data Warehouse Architecture Best Practices

1. Business-Driven Design: Align warehouse goals with strategic KPIs; avoid "build it and they will come" syndrome.

2. Scalable Architecture: Implement three-tier architecture (staging, data warehouse, data marts) for separation of concerns and scalability.

3. Dimensional Modeling: Use Ralph Kimball's star schemas with conformed dimensions for query performance and business user comprehension.

4. ETL Framework: Develop metadata-driven, reusable ETL components with built-in data quality checks and audit trails.

5. Slowly Changing Dimensions: Implement Type 2 SCDs for tracking historical changes to critical dimensions (customers, products).

6. Index Strategy: Use bitmap indexes for low-cardinality columns; materialized views for common aggregations; partition tables by date.

7. Security by Design: Row-level security (RLS), column-level encryption for PII, and role-based access from inception.

8. Change Management: Version control for ETL code, schema change procedures, and rollback capabilities.

9. Monitoring & Alerting: Track ETL performance, data freshness, and quality metrics via dashboards. Implement SLA alerts.

10. Documentation: Maintain comprehensive business glossary, data lineage diagrams, and ETL process documentation for knowledge transfer.

11. Cloud-Native Design: Leverage serverless computing, auto-scaling, and managed services to optimize cost-performance.

---

20. Web Mining: Tasks, Features, Applications

Web Mining is the application of data mining techniques to discover patterns from web documents, server logs, and hyperlinks.

Tasks:

1. Web Content Mining: Extract knowledge from page content (text, images, video) using text mining and IR techniques.
2. Web Structure Mining: Analyze hyperlink topology to discover authoritative pages (PageRank) and community structures.
3. Web Usage Mining: Analyze user interaction data (logs, clickstreams) for behavior patterns.

Features:
- Scale: Billions of documents and terabytes of log data daily
- Heterogeneity: Structured (databases), semi-structured (HTML/XML), unstructured (text)
- Dynamicity: Content updates frequently; temporal patterns matter
- Rich Linkage: Hyperlinks provide citation analysis opportunities
- Noisy Data: Logs contain bots, missing references, incomplete sessions

Applications:
- Personalization: Amazon's recommendation engine based on browsing history
- Fraud Detection: Identifying click fraud in online advertising
- Search Engine Optimization: Analyzing query logs to improve ranking
- Crawl Optimization: Prioritizing high-value pages for web crawlers
- Social Network Analysis: Discovering influence patterns and communities
- E-commerce: Shopping cart analysis and churn prediction

---

21. Data Preprocessing Stages

Comprehensive Pipeline:

1. Data Cleaning:
Handle missing values (imputation/deletion), detect and smooth noisy data (outlier removal), resolve inconsistencies (standardize formats), and handle duplicates (record linkage). *Tools:* OpenRefine, custom Python scripts.

2. Data Integration:
Merge data from multiple sources, resolving schema conflicts (naming, structural), entity identification (same customer across systems), and value conflicts (different units). *Challenges:* Redundancy, heterogeneity, semantic heterogeneity.

3. Data Reduction:
Reduce volume without sacrificing analytical quality:
- Dimensionality Reduction: PCA, feature selection
- Numerosity Reduction: Sampling, data cube aggregation
- Data Compression: Wavelet transforms, principal component compression

4. Data Transformation:
Convert data into appropriate forms for mining:
- Normalization: Scale to [0,1] or z-score
- Discretization: Bin continuous attributes
- Concept Hierarchy Generation: Generalize attributes (e.g., street → city → state)
- Attribute Construction: Derive new features

---

22. Data Warehouse Design Challenges

Key Challenges:

1. Requirements Elicitation: Business users often can't articulate analytical needs upfront; requirements evolve. Solution: Agile, iterative development with prototypes.

2. Source System Variability: Legacy systems lack documentation; schemas change without notice. Solution: Robust ETL error handling and metadata management.

3. Data Quality: "Garbage in, garbage out." Inconsistent, incomplete, duplicate data across sources. Solution: Implement data quality dashboards and stewardship programs.

4. Scalability: Exponential data growth strains performance. Solution: Horizontal partitioning, columnar storage, MPP architectures (Snowflake, Redshift).

5. Real-Time Integration: Traditional batch ETL can't support operational analytics. Solution: Lambda architecture with streaming pipelines (Kafka, Flink).

6. Security & Compliance: Balancing data democratization with GDPR, HIPAA, CCPA requirements. Solution: Dynamic data masking, attribute-based access control.

7. Organizational Resistance: Siloed departments resist sharing data. Solution: Executive sponsorship, data governance council with cross-functional representation.

8. Cost Management: Cloud storage/compute costs can spiral. Solution: Lifecycle policies, resource tagging, and chargeback models.

9. Technology Selection: Rapidly evolving ecosystem creates decision paralysis. Solution: Focus on business value over hype; prefer interoperable standards.

10. Change Management: Evolving business rules require schema changes. Solution: Versioned schemas, blue-green deployments, and impact analysis tools.

---

23. Data Lake vs. Data Warehouse

Dimension	Data Lake	Data Warehouse
Data Structure	Raw, unprocessed; schema-on-read	Processed, curated; schema-on-write
Data Types	All (structured, semi-, unstructured)	Primarily structured (tables)
Schema	Flexible, evolves; no upfront design	Rigid, predefined; designed upfront
Users	Data scientists, ML engineers	Business analysts, BI professionals
Use Case	Exploration, ML, deep learning	Reporting, dashboards, OLAP
Performance	Ad-hoc queries slower (full scans)	Optimized for fast aggregations
Cost	Lower storage cost (object storage)	Higher cost (proprietary appliances)
Quality	"Swamp" risk without governance	High quality, trusted, curated
Example Tech	Hadoop, S3, Spark, Delta Lake	Oracle, Teradata, Snowflake

Synergy: Modern architectures use both—lakes store raw data for exploration; warehouses contain refined aggregates for enterprise reporting. Tools like Databricks Delta Lake bridge the gap.

---

24. Apriori Algorithm

Purpose: Efficiently find frequent itemsets and generate association rules without exhaustive search.

Algorithm Steps:

```
1. Scan database to find frequent 1-itemsets (L_1) with support ≥ minsup
2. k = 1
3. While L_k ≠ ∅:
   a. Candidate Generation: Join L_k with itself to generate C_{k+1} (candidate (k+1)-itemsets)
   b. Pruning: Remove candidates with infrequent subsets using Apriori property:
      "All subsets of a frequent itemset must be frequent"
   c. Scan database to count support for C_{k+1}
   d. L_{k+1} = {c ∈ C_{k+1} | support(c) ≥ minsup}
   e. k = k + 1
4. Return ∪ L_k
```

Example Execution:
Database: {1,2,3}, {1,2,4}, {1,3,4}, {2,3,4}, {2,3}
minsup = 60% (≥3 transactions)

- L_1: {1:3, 2:4, 3:4, 4:3} (all frequent)
- C_2: {12,13,14,23,24,34}
- L_2: {12:2}→infrequent, {13:2}→infrequent, {14:2}→infrequent, {23:3}→frequent, {24:2}→infrequent, {34:2}→infrequent
- C_3: {123} (pruned because 12,13 are infrequent)
- L_3: ∅ → Terminate

Optimization: Use hash trees for fast subset checking; partition database for parallel scanning.

25. Cluster Analysis in Data Mining

Cluster Analysis groups unlabeled objects into cohesive clusters where intra-cluster similarity is maximized and inter-cluster similarity is minimized.

Use in Data Mining:
- Exploratory Data Analysis: Discover natural groupings (customer segments, gene expression patterns)
- Preprocessing: Dimension reduction; outlier detection (points far from all clusters)
- Summarization: Represent clusters by centroids for compression
- Pattern Recognition: Image segmentation, document categorization

Example: Retail customer segmentation using RFM (Recency, Frequency, Monetary) values. K-means identifies "High-Value Loyalists," "At-Risk Customers," and "Bargain Hunters" for targeted marketing.

Applications:
- Biology: Taxonomy creation, gene clustering
- Marketing: Customer segmentation for personalized campaigns
- Web: Clustering search results by topic
- Image Processing: Segmentation, object recognition
- Anomaly Detection: Network intrusion detection (clusters = normal behavior; outliers = attacks)
- Recommendation Systems: Cluster users with similar tastes

---

26. Dimensionality Reduction: Feature Selection vs. Extraction

Feature Selection: Selects a subset of original features, preserving semantics. Methods:
- Filter: Correlation, chi-square test, mutual information (independent of model)
- Wrapper: Use model performance as selection criterion (computationally expensive)
- Embedded: Lasso (L1 regularization) automatically selects features during model training

Advantages: Interpretability, no transformation overhead, preserves meaning.
Example: Selecting {income, credit_score, age} from 50 features for loan prediction.

Feature Extraction: Creates new features as combinations of originals, transforming to lower-dimensional space:
- PCA (Linear): Projects onto principal components (directions of maximum variance)
- LDA (Supervised): Maximizes class separability
- t-SNE/Autoencoders (Non-linear): Capture complex manifolds

Advantages: Can capture latent patterns; better for high-dimensional correlated data.
Example: PCA on 1000-gene expression data → 10 principal components capturing 95% variance.

Trade-off: Selection maintains interpretability; extraction can achieve better compression but loses direct meaning.

---

27. Layered ETL Implementation

Three-Layer Architecture:

Layer 1: Staging Area (Bronze)
- Purpose: Raw data landing zone from sources

- Characteristics: Schema mirrors sources; minimal transformation; short retention (7-30 days)
- Activities: Data extraction, basic validation, CDC capture
- Storage: Flat files (Parquet) or temporary tables
- Tools: Apache NiFi, AWS DMS

Layer 2: Integration Layer (Silver)
- Purpose: Cleaned, conformed, integrated data following enterprise standards
- Characteristics: Normalized form; surrogate keys; referential integrity; quality-checked
- Activities: Data cleaning, deduplication, mastering (MDM), SCD processing, integration
- Storage: Persistent staging tables or ODS (Operational Data Store)
- Tools: Informatica, Talend, Spark

Layer 3: Presentation Layer (Gold)
- Purpose: Optimized for query performance and business consumption
- Characteristics: Denormalized star schemas; aggregated summary tables; materialized views
- Activities: Aggregation, denormalization, creation of data marts
- Storage: Warehouse fact/dimension tables, OLAP cubes
- Tools: SQL, DBT, MicroStrategy

Benefits: Separation of concerns; failure isolation; data lineage clarity; incremental processing.

---

28. OLAP: Multicube vs. Hypercube

OLAP (Online Analytical Processing) enables multidimensional analysis using data cube abstractions.

Multicube (MOLAP - Multidimensional OLAP):
- Structure: Data stored in pre-computed multidimensional arrays (cubes) on disk
- Characteristics: Optimized for fast slice/dice; requires pre-aggregation; sparse data wastes space
- Implementation: Proprietary formats (Essbase, Cognos PowerCubes)
- Query: OLAP server handles MDX queries; performs in-memory operations
- Advantage: Sub-second response for pre-aggregated data
- Disadvantage: "Data explosion" problem; limited scalability

Hypercube (HOLAP - Hybrid OLAP):
- Structure: Combines ROLAP base (relational storage) with MOLAP aggregates for hot data
- Characteristics: Best of both worlds; detailed data in RDBMS; summary cubes for performance
- Implementation: SQL Server Analysis Services, SAP BW
- Query: Route summary queries to MOLAP; drill-down to ROLAP
- Advantage: Scalable storage + fast aggregations; flexible partitioning
- Disadvantage: Complex query routing; potential consistency issues

Comparison:
- Storage: Multicube=dedicated arrays; Hypercube=hybrid RDBMS+arrays
- Performance: Multicube faster for summarized data; Hypercube balances performance with scalability
- Use Case: Multicube for fixed reporting; Hypercube for exploratory analysis with varied granularity

---

29. Binning Smoothing: Equal-Frequency Example

Problem: Delete noise from set: {4, 2, 6, 10, 8, 16, 12, 24, 22, 14, 26} using equal-frequency binning (3 bins) with mean smoothing.

Solution:

Step 1: Sort Data
{2, 4, 6, 8, 10, 12, 14, 16, 22, 24, 26}

Step 2: Partition into 3 Equal-Frequency Bins (11 points → 4, 4, 3 points per bin)

- Bin 1: {2, 4, 6, 8}
- Bin 2: {10, 12, 14, 16}
- Bin 3: {22, 24, 26}

Step 3: Calculate Bin Means

- $\text{Mean}_1 = (2+4+6+8)/4 = 20/4 = 5$
- $\text{Mean}_2 = (10+12+14+16)/4 = 52/4 = 13$
- $\text{Mean}_3 = (22+24+26)/3 = 72/3 = 24$

Step 4: Smooth by Replacing with Bin Means

Smoothed Set: {5, 5, 5, 5, 13, 13, 13, 13, 24, 24, 24}

Result: Original "noise" (e.g., irregular jumps between values) is replaced with representative bin averages, preserving overall distribution while eliminating granularity. The steep jump from 8→10 (Bin1→Bin2) and 16→22 (Bin2→Bin3) remains, representing true distributional boundaries.

---

30. K-Nearest Neighbors (K-NN) Classification

Algorithm:
```
1. Given training set D = {(x_i, y_i)} and query point q
2. Compute distance d(q, x_i) for all training points (typically Euclidean)
3. Select k nearest neighbors: N_k(q) = {k points with smallest d}
4. Predict class: ŷ = majority_vote({y_i | x_i ∈ N_k(q)})
5. (For regression: ŷ = average of neighbor values)
```

Example: Classify new customer (Age=35, Income=$60k) as {Default, No Default} with k=3. Find 3 nearest historical customers; if 2 are "No Default," predict "No Default."

Advantages:
- Non-parametric: No assumptions about data distribution
- Lazy Learning: No training phase; adapts to new data instantly
- Simple: Intuitive and easy to implement
- Versatile: Works for classification and regression

Disadvantages:
- Computational Cost: O(n×d) for each query; impractical for large n
- Memory Intensive: Stores entire training set
- Curse of Dimensionality: Distance metrics degrade in high dimensions
- Sensitive to Scale: Requires feature normalization
- Optimal k Selection: Small k → overfitting; large k → underfitting; requires cross-validation

Optimization: Use KD-trees, ball trees, or locality-sensitive hashing for faster neighbor search; use weighted voting by distance.

---

31. Vector Space Model & TF-IDF

Vector Space Model (VSM):
Represents documents as vectors in high-dimensional space where dimensions correspond to terms. Document similarity is computed via cosine similarity:
```
cos(d1, d2) = (d1·d2) / (||d1|| × ||d2||)
```

TF-IDF (Term Frequency-Inverse Document Frequency):
Weights terms by importance: frequent in document but rare in corpus.

Components:

Term Frequency (TF): Measures term importance within a document.
```
TF(t,d) = 1 + log(freq(t,d))  if freq(t,d) > 0
          0                    otherwise
```

Inverse Document Frequency (IDF): Measures term rarity across corpus.
```
IDF(t) = log(N / df(t) + 1)  (smoothing prevents division by zero)
```

TF-IDF Weight:
```
TF-IDF(t,d) = TF(t,d) × IDF(t)
```

Complete Example:
Corpus: D_1="data mining", D_2="data warehouse", D_3="mining warehouse"
N=3

For term "data" in D_1:
- TF("data",D_1) = 1 + log(1) = 1
- df("data") = 2 (D_1, D_2)
- IDF("data") = log(3/2+1) ≈ log(2.5) ≈ 0.92
- TF-IDF = 1 × 0.92 = 0.92

For term "mining" in D_1:
- TF("mining",D_1) = 1
- df("mining") = 2 (D_1, D_3)
- IDF("mining") = log(3/2+1) ≈ 0.92
- TF-IDF = 0.92

Result: D_1 vector ≈ [0.92 (data), 0.92 (mining), 0 (warehouse)]

Interpretation: "data" and "mining" are equally important in D_1; "warehouse" is irrelevant.

---

32. Data Integration Issues

Schema Integration:
- Problem: Homonyms (same name, different semantics: "ID" = customer ID vs. product ID) and synonyms (different names, same semantics: "CustNo" vs. "Customer_ID")
- Solution: Enterprise data dictionary, semantic mapping tables, ontology-based integration

Redundancy:
- Problem: Same attribute stored multiple times across sources, causing storage waste and inconsistency
- Detection: Correlation analysis (χ^2 test for categorical; covariance for numeric)
- Resolution: Master Data Management (MDM); deduplication using record linkage (fuzzy matching on name/address); create "system of record" definitions

Data Value Conflicts:
- Problem: Different representations: Date (MM/DD/YYYY vs. DD-MM-YYYY), Currency ($1000 vs. 1000 USD), Units (kg vs. lbs), and semantic differences (revenue recognition policies)
- Detection: Statistical profiling (range checks, pattern matching)
- Resolution: Standardization rules in ETL; use conversion functions; establish "golden record" through conflict resolution policies (most recent, most trusted source)

---

33. LDA vs. PCA Feature Extraction

Linear Discriminant Analysis (LDA):
- Type: Supervised linear transformation
- Objective: Maximize class separability by projecting onto directions that maximize between-class variance while minimizing within-class variance
- Goal: Find subspace where classes are best separated
- Mathematical Criterion: Maximize $J(w) = (w^T S_B w) / (w^T S_W w)$
  - S_B = between-class scatter matrix
  - S_W = within-class scatter matrix
- Output: At most c-1 dimensions (c = number of classes)
- Use Case: Classification preprocessing when labels are available; face recognition (Fisherfaces)

Principal Component Analysis (PCA):
- Type: Unsupervised linear transformation
- Objective: Maximize variance in projected subspace; finds directions of maximum data spread
- Goal: Dimensionality reduction while preserving information
- Mathematical Criterion: Maximize variance = $w^T \Sigma w$ where Σ = covariance matrix
- Output: Eigenvectors (principal components) sorted by eigenvalues
- Use Case: Exploratory analysis; noise reduction; visualization; when labels are unavailable

Key Differences:
- Supervision: LDA uses labels; PCA is unsupervised
- Goal: LDA seeks discrimination; PCA seeks representation
- Dimensions: LDA limited to c-1; PCA limited by rank of data matrix
- Example: For medical diagnosis, LDA is superior as it uses disease labels to find discriminative biomarkers; PCA would just find most variant genes regardless of diagnostic utility.

This completes the theoretical framework. Each answer provides the depth expected in graduate-level examinations, balancing mathematical rigor with practical applicability. Students should supplement these concepts with hands-on implementation using Python/R tools to solidify understanding.

I'll provide comprehensive, professor-level answers for each of these data architecture and mining concepts with detailed textual diagrams and explanations.

MOLAP & HOLAP Architecture

Multidimensional OLAP (MOLAP) stores data in a native multidimensional cube structure, pre-aggregating data for fast query performance. The architecture consists of:

```
[Data Sources] → [ETL Process] → [MOLAP Server]
          ↓
     [Multidimensional Database (Cube)]
          ↓
     [OLAP Client/Analysis Tools]
```

Hybrid OLAP (HOLAP) combines MOLAP and ROLAP (Relational OLAP) benefits, storing detailed data in relational tables and aggregations in cubes:

```
[Data Sources] → [ETL Process] → [HOLAP Engine]
          |               |
          ↓               ↓
     [Relational DB]   [MOLAP Cube]
          \         /
           \       /
            ↓     ↓
       [Unified Query Interface]
              ↓
       [OLAP Client Tools]
```

Professor's Note: MOLAP excels in performance but suffers from data explosion and size limitations. HOLAP provides a pragmatic compromise—relational storage offers scalability for atomic data, while multidimensional cubes accelerate aggregation queries. The trade-off is increased architectural complexity.

Decision Tree Construction

Using the classic "Play Tennis" dataset with attributes: Outlook (Sunny, Overcast, Rain), Humidity (High, Normal), Wind (Weak, Strong), and target: Play (Yes, No).

Construction Process:

1. Select Root Attribute: Calculate information gain. Outlook provides highest gain (0.246 bits)
2. Split Data: Create branches for each Outlook value
3. Recursive Partitioning: For Sunny branch (entropy remains high), split on Humidity
4. Stopping Criteria: When node is pure or gain < threshold

Tree Representation:

```
          Outlook
         /  |  \
     Sunny Overcast Rain
       |     |     |
    Humidity   Yes  Wind
    /   \          / \
  High  Normal    Weak Strong
   |     |        |    |
   No    Yes      Yes   No
```

Key Algorithms: ID3 uses information gain; C4.5 uses gain ratio; CART uses Gini impurity. The tree above would be generated by any algorithm as Overcast perfectly predicts "Yes."

Star Schema

Consider a retail sales data warehouse with Fact_Sales (grain: each line item) and dimensions:

Example Structure:

```
            FACT_SALES
     (Sales_ID, DateKey, ProductKey, StoreKey, CustomerKey,
      Quantity, Unit_Price, Discount, Sales_Amount)
         ↑       ↑       ↑       ↑       ↑
         |       |       |       |       |
   +------+----------+---------+---------+---------+------+
   |    |       |      |      |            |
  DIM_DATE  DIM_PRODUCT  DIM_STORE  DIM_CUSTOMER     DIM_PROMOTION
```

Detailed Diagram:

```
            [Fact_Sales]
   ┌─────────────┬──────────┬──────────┬──────────┬──────────┐
   │ DateKey │ProductKey│ StoreKey │CustomerKey│
```



```
   │ (FK)   │ (FK)   │ (FK)   │ (FK)   │
   │ Quantity │ UnitPrice │ Discount │ SalesAmt │
   └─────┬─────┬─────┬─────┬─────┬─────┬─────┘
      │     │     │     │
 ┌─────────────▼──┐ ┌────▼────┐ ┌──▼──┐ ┌──────▼──┐
 │ Dim_Date    │ │Dim_Prod │ │Dim_S│ │Dim_Cust│
 │ DateKey (PK) │ │ProdKeyPK│ │Store│ │CustKey │
 │ FullDate   │ │ ProdName│ │Name │ │ Name  │
 │  Year    │ │ Category│ │City │ │ City  │
 │ Quarter   │ │ Brand  │ │State│ │ Segment│
 └───────────────┘ └─────────┘ └─────┘ └────────┘
```

Characteristics: Denormalized dimensions, simple joins, fast query performance, high storage redundancy. This is the preferred schema for most BI applications due to its intuitive structure matching business perspectives.

Multidimensional Schema & Anchor Modeling

Multidimensional Schema represents data as cubes with dimensions and measures. The data cube is the logical representation:

```
          Time
           ↑
           |
  Location → Sales Cube ← Product
           |
           ↓
       Measures (Sales, Profit)
```

Anchor Modeling is a sixth-normal-form approach for temporal data warehousing. It uses three construct types:

1. Anchors: Core business entities (identifiers only)
2. Attributes: Time-variant properties tied to anchors
3. Ties: Relationships between anchors

Example: Customer-Order-Product

```
[ANC_Customer] ←─┬─ [ATTR_CustomerName] (temporal)
  (CustomerID)   └─ [ATTR_CustomerEmail] (temporal)
     ↑
     │ (ties)
     ▼
[TIE_CustomerOrder] ──→ [ANC_Order] ←─ [TIE_OrderProduct] ──→ [ANC_Product]
  (CustomerID,OrderID)   (OrderID)     (OrderID,ProductID)   (ProductID)
                ↑
                │
          [ATTR_OrderDate]
```


Visualization: Imagine a constellation where anchors are stars, attributes are orbiting planets (with timestamps), and ties are gravitational connections. This handles schema evolution elegantly—adding attributes never alters existing tables.

Data Warehouse Architecture Components

Enterprise Data Warehouse Architecture:

```
┌──────────────────────────────────────────────────────────┐
│       Data Sources Layer           │
│ ┌─────┐ ┌─────┐ ┌─────┐ ┌─────┐ ┌─────┐       │
│ │OLTP │ │ERP  │ │ CRM │ │Web  │ │Exter│ │     │       │
│ └─────┘ └─────┘ └─────┘ └─────┘ └─────┘       │
└──────────────┬───────────────────────────────────────────┘
     │
     ▼
┌──────────────────────────────────────────────────────────┐
│       ETL Layer (Staging)          │
│ [Extract → Clean → Transform → Aggregate → Load]   │
│ ┌─────────┐ ┌────────┐ ┌─────────┐ ┌────────────┐  │
│ │Staging  │→│Scrubber│→│Integrator│→│Transformer │ │  │
│ └─────────┘ └────────┘ └─────────┘ └────────────┘  │
└──────────────┬───────────────────────────────────────────┘
     │
     ▼
┌──────────────────────────────────────────────────────────┐
│     Enterprise Data Warehouse (EDW)      │
│ ┌──────────────────────────────────────┐     │
│ │ Integrated, Subject-Oriented Data │←─────┐ │  │
│ └──────────────────────────────────────┘  │   │
└────────────┬─────────────────────────────────┬─────────┘
    │
    └───────────┬──────────────────┬────────────────
       │      │
 ┌───────────▼──┐ ┌──────────▼──────────┐
 │ Metadata   │ │ Data Marts  │
 │ Repository │ │ (Departmental) │
 └──────────────┘ └──────────┬──────────┘
       │      │
 ┌───────────▼────────┐ ┌────────▼────────┐
 │ Operational Meta │ │ Dim_Models │
 │ Technical Meta  │ │ Aggregates │
 │ Business Meta  │ └─────────────────┘
 └────────────────────┘
       │
 ┌───────────▼───────────┐
 │ Access Tools Layer │
 │ (Query, OLAP, DM)  │
 └───────────┬───────────┘
       │
 ┌───────────▼───────────┐
```

```
    | Reporting Layer        |
    | (Dashboards, Reports)  |
```

Component Explanations:
- ETL: Orchestrates data movement; includes data quality, lineage, and scheduling
- Metadata: Three types—technical (schemas), business (definitions), operational (load stats)
- Access Tools: Enable query (SQL), analysis (OLAP), and discovery (data mining)
- Reporting Layer: Presentation-tier with parameterized reports, alerts, and visualization

---

Snowflake Schema

Use-Case: Global electronics retailer with complex product hierarchies and many stores.

When to Use: When dimensions are very large, highly normalized storage is critical, or hierarchical relationships are deep and require separate maintenance.

Example Diagram:

```
          FACT_SALES
 | DateKey | ProductKey | StoreKey | CustomerKey |

 | Dim_Date |  | Dim_Prod |  | Dim_ |  | Dim_Custom |
   (PK)          (FK)Cat      Stor

        | Dim_Catego |  | Dim_ |
        | gory       |  | City |

        | Dim_Subcat |  | Dim_Reg |
```

Key Difference: Product dimension is normalized into cascading tables (Product → Category → Subcategory). While storage is optimized, queries require more joins, impacting performance. The snowflake "branches" create a crystalline structure.

Hadoop Architecture for Data Warehousing

Conceptual Architecture:


```
          Cluster Management
          [YARN ResourceManager]

 Node 1             Node 2              Node N
 [NodeManager]      [NodeManager]       [NodeMgr]

 | DataNode |       | DataNode |        | DataNode |
 | (HDFS)   |       | (HDFS)   |        | (HDFS)   |

 | Container  |     | Container  |      | Contain |
 | (Map/Red)  |     | (Map/Red)  |      | (Task)  |

        Distributed HDFS Storage

        | Hive / Impala   |
        | (SQL Interface) |

        | HBase (NoSQL) |
        | (Real-time)   |

        | ETL Framework  |
        | (Sqoop, Flume, |
        | Spark, NiFi)   |
```

Professor's Commentary: This architecture implements schema-on-read philosophy. Data lands in HDFS raw, then is progressively structured through ETL pipelines. Hive provides data warehouse semantics on commodity hardware, enabling petabyte-scale analytics previously impossible with traditional MPP systems.

Star & Snowflake Diagrams (Case Study)

For Sales-Items Fact with Products, Customers, Time, Locations:

Star Schema:


```
        FACT_SALES_ITEMS
 | TimeKey  | ProductKey | CustomerKey | LocationKey |
 | Quantity | UnitCost   | Discount    | Revenue     |

 | Dim_Time    | Dim_Produc | Dim_Cus  | Dim_Locat  |
 | (PK)TimeKey | ProductKey | tomerKe  | LocationK  |
 | FullDate    | Name       | Name     | City       |
 | YearQtr     | Category   | Segment  | Country    |
 | MonthNum    | Brand      | Tier     | Region     |
```

Snowflake Schema:

```
        FACT_SALES_ITEMS
 | TimeKey | ProductKey | CustomerKey | LocationKey |

 | Dim_Time |  | Dim_Produc |  | Dim_Cus |  | Dim_Locat |
              | ProductKey |  | tomerKe |  | LocationK |
              | Name       |

      | Dim_Catego |       | Dim_Country |
      | (FK)       |       | (FK)        |
```

Analysis: The star schema denormalizes Category into Product and Country into Location. The snowflake normalizes these into separate tables. For agile BI, star is superior; for massive dimensions (millions of products), snowflake reduces update anomalies when categories restructure.

ETL Process Diagram

Detailed ETL Pipeline:

```
        EXTRACTION LAYER
 | Source A |  | Source B      |  | Source C |
 | (Full)   |  | (Incremental) |  | (CDC)    |
```



```
 | Staging Area (Raw, Unchanged) |

        TRANSFORMATION LAYER
 | Cleanse | → | Validate | → | Resolve | → | Integrat | → | Aggre |  | gate |
        | Business Rules Engine |

        LOADING LAYER
 | Initial |  | Increment |  | Slowly     |
 | Load    |  | Load      |  | Dim Update |

 | Target Data Warehouse |
```

Detailed Steps:
1. Extract: Full pull for initial load; CDC (Change Data Capture) or timestamp-based for deltas
2. Cleanse: Handle missing values, outliers, standardize formats (e.g., dates to YYYY-MM-DD)
3. Validate: Referential integrity checks; business rule validation
4. Resolve: Deduplicate records; survivorship rules for golden records
5. Integrate: Conform dimensions across sources; surrogate key assignment
6. Aggregate: Pre-compute summary tables for performance
7. Load: Use bulk loading; maintain transaction integrity; handle SCDs (Slowly Changing Dimensions)

Real-Time Data Warehouse Architecture

```
        Real-Time Data Sources
 [OLTP → Transaction Log]  [Sensors]  [Web Logs]  [Apps]

        Stream Processing Layer
 [Kafka / Kinesis → Spark Streaming]
            ↓
 [Micro-batch Extraction]
```



```
        Real-Time ETL (Hot Path)
 | Parse | → | Validate | → | Enrich | → | Load RT Mart |

 | Real-Time  |  | Traditional   |
 | Data Mart  |  | DW            |
 | (MemSQL/   |  | (Daily Batch) |
 | Druid)     |

        Unified Query Layer
 [BI Tool → Federation Layer]

        Real-Time Dashboards & Alerts
```

Architecture Pattern: Lambda Architecture with dual paths—hot (real-time) and cold (batch). The real-time path uses stream processing for near-instant insights (<5 seconds latency), while the batch path ensures data quality and historical accuracy. The serving layer federates both.

Fact Constellation Schema

Definition: A collection of multiple fact tables sharing conformed dimensions—also called Galaxy Schema. Unlike star/snowflake (single fact), it represents complex business processes.

Comparison to Snowflake:
- Snowflake: Normalizes ONE fact table's dimensions
- Constellation: Multiple fact tables with SHARED dimensions

Example: Retail Enterprise

```
 Dim_Product ←
 Dim_Customer ←
```



```
 Dim_Time ←
     | Fact_Sales (Transaction)     |
     | (Quantity, Price, Revenue)   |

     | Dim_Store |

 Dim_Supplier ←
 Dim_Time ←
     | Fact_Inventory                    |
     | (Units_On_Hand, WIP, Shrink)      |
```

Characteristics: Fact tables at different grains (transaction vs. daily snapshot) share common dimensions. This models real enterprise complexity where you analyze sales, inventory, and procurement as interconnected processes rather than silos.

Data Mart Structure

Example: Marketing Data Mart focusing on campaign effectiveness.

```
        MARKETING DATA MART
  | Fact_Campaign_Response (Campaign Grain) |
  | CampaignKey | CustomerKey | ResponseDate |
  | ResponseType| ChannelKey  | Cost         |

  | Dim_Campaign |  | Dim_Customer  |  | Dim_Channel    |
  | CampaignName |  | Demographics  |  | ChannelType    |
  | StartDate    |  | Segment       |  | CostPerLead    |
  | Budget       |  | LifetimeValue |  | Attribution    |

  | Derived Tables: Campaign_ROI, Attribution_Paths |
```


Data Mart Characteristics:
- Dependent: Sourced from enterprise DW (top-down)
- Independent: Direct from sources (bottom-up)
- Hybrid: Combination approach

The structure includes summary tables for common queries, reducing compute at runtime. This is a non-federated, physically separate mart optimized for marketing analytics.

OLAP Operations: Roll-up & Drill-down

Data Cube Context: Sales data by Product, Time, Location.

Roll-up (Consolidation): Summarizes data along dimension hierarchies.

Before Roll-up:

Product	Month	City	Sales
Laptop	Jan 2024	New York	50,000
Laptop	Jan 2024	Boston	30,000
Mouse	Jan 2024	New York	5,000

After Roll-up (City → Region, Product → Category):

Category	Month	Region	Sales
Computers	Jan 2024	Northeast	80,000
Accessories	Jan 2024	Northeast	5,000

Drill-down (Explosion): Navigates from summary to detail.

Before Drill-down:

Quarter	Region	Sales
Q1 2024	Northeast	255,000

After Drill-down (Quarter → Week → Day, Region → Store):

Week	Store	Day	Sales
Week 1	Store A	Jan 1	2,500
Week 1	Store A	Jan 2	3,200
Week 1	Store B	Jan 1	1,800

Geometric Interpretation: Imagine a Rubik's cube. Roll-up removes a layer, collapsing the cube. Drill-down adds a layer, expanding dimensionality. Operations occur along hierarchies defined in dimension tables.

Vector Space Modeling for Text

Geometric Representation: Documents as vectors in high-dimensional term space.

```
   Term2 (Machine)
     ↑
     |   d2: "Learning Machine"
     |  ↗
     | /
     |/
     └──────────→ Term1 (Learning)
    /
   /  d1: "Deep Learning"
  ↙
 d3: "Deep Machine"
   (Origin = Term3: Deep)
```

Formal Model:
- Vocabulary: {learning, machine, deep, algorithm, ...} = dimensions
- Document Vector: $d = (w_1, w_2, w_3, ..., w_n)$ where w_i = TF-IDF weight
- TF-IDF: $w_{ij} = tf_{ij} \times \log(N/df_i)$

Similarity: Cosine similarity measures angle between vectors:
$\cos(d_1, d_2) = (d_1 \cdot d_2) / (||d_1|| \times ||d_2||)$

Professor's Insight: This models semantic similarity geometrically. Orthogonal vectors = unrelated documents. The curse of dimensionality requires dimensionality reduction (LSI, LDA) for practical applications.

Cluster Analysis & Visualization

Definition: Unsupervised partitioning of data into groups (clusters) where intra-cluster similarity is maximized and inter-cluster similarity is minimized.

In Data Mining: Used for segmentation, anomaly detection, pattern recognition.

2D Visualization with Centroids:

```
Feature 2 (Income)
   ↑
60 |    • •
   |  •   •       • •
   | •     •     •   •
   |•       •   •     •   ▲ Cluster 2
   |•  C1  •   •  C2 •   | (Centroid)
30 |————————————•———•————————•———▲
   |      • •        •   | Cluster 1
   |     •  •       •    | (Centroid)
   |    •    •     •     |
   |   •     •    •      |
 0 └──────────────────────────────→ Feature 1 (Age)
   0       30      60
```

Algorithms: K-means (partitioning), DBSCAN (density-based), Hierarchical (dendrograms). Each produces different cluster shapes—K-means assumes spherical clusters; DBSCAN finds arbitrary shapes.

Web Structure Mining

Types of Web Mining:
1. Web Content Mining: Extracting knowledge from page content (text, images)
2. Web Usage Mining: Discovering patterns from server logs (clickstreams)
3. Web Structure Mining: Analyzing link graphs

Structure Mining Graph:

```
    Hub Pages (Yahoo, DMOZ)
       ↑   ↑   ↑
       |   |   |
       ↓   ↓   ↓
    Authority Pages (High In-degree)
       ↑
       |
       ▼
    ┌──────────────┐
    │  Web Graph   │
    │ (Directed)   │
    │ Adjacency    │
    │  Matrix      │
    └──────┬───────┘
           |
           ▼
    [PageRank Algorithm]
           |
           ▼
    Authority Scores
```


HITS Algorithm: Distinguishes Hubs (pages that link to many authorities) from Authorities (pages linked by many hubs). Used in search engines and social network analysis. The web is modeled as a directed graph G = (V, E) where V = pages, E = hyperlinks.

Data Lake Stages

Evolution from Puddle to Ocean:

```
┌─────────────────────────────────────────────────────┐
│ DATA PUDDLE (Project Silo)               │
│ ┌──────────────────────────────┐            │
│ │ Single Department, Limited Scope  │      │
│ │ Size: GBs, Structure: Known  │      │
│ └──────────────────────────────┘            │
└─────────────────────┬───────────────────────────────┘
          ↓
┌─────────────────────────────────────────────────────┐
│ DATA POND (Multi-Project)                │
│ ┌──────────────────────────────┐            │
│ │ Several Departments, Some Integration│      │
│ │ Size: TBs, Structure: Semi-structured│      │
│ └──────────────────────────────┘            │
└─────────────────────┬───────────────────────────────┘
          ↓
┌─────────────────────────────────────────────────────┐
│ DATA LAKE (Enterprise-Wide)              │
│ ┌──────────────────────────────┐            │
│ │ All Enterprise Data, Governed  │      │
│ │ Size: PBs, Structure: Raw + Curated  │      │
│ │ Zones: Raw, Staging, Analytics, Sandbox│      │
│ └──────────────────────────────┘            │
└─────────────────────┬───────────────────────────────┘
          ↓
┌─────────────────────────────────────────────────────┐
│ DATA OCEAN (Ecosystem)                   │
│ ┌──────────────────────────────┐            │
│ │ Multi-Enterprise, IoT, External APIs │      │
│ │ Size: Exabytes, Structure: Polyglot │      │
│ │ Global governance, AI-driven catalog │      │
│ └──────────────────────────────┘            │
└─────────────────────────────────────────────────────┘
```

Zones Within a Lake:
- Raw: Immutable source copies (bronze)
- Cleansed: Validated, cleaned data (silver)
- Curated: Business-ready aggregations (gold)
- Sandbox: Experimental area

K-Means Clustering Algorithm

Algorithm Steps:

```
Initialize: Randomly place k centroids in feature space
Repeat until convergence:
   Assignment: Assign each point to nearest centroid
            (using Euclidean distance)
   Update: Recalculate centroids as mean of assigned points
```

Iterative Process Visualization:

```
Iteration 0 (Random Centroids)   Iteration 1 (Assignment)   Iteration N (Converged)
   •                      •                 •
  • •                    • •               • •
•    •    C1              •    •  C1         •    •  C1
   •                     •                 •
     •                    •                 •
                         • |               • |
                          |                 |
   •                       |                 |
    •                      |                 |
     •    C2                 |  C2               |  C2
      •                    |                 |
                         •/                •/
```

Pseudocode:

```python
# Professor's Formal Description
function K-MEANS(D, k):
   # D: dataset, k: number of clusters
   C ← initialize_k_random_centroids(D)
   repeat:
      # Assignment step
      for each point p in D:
         cluster_id[p] ← argmin_i distance(p, C[i])

      # Update step
      for each cluster i in 1..k:
         C[i] ← mean({p | cluster_id[p] = i})
   until C stops changing
   return cluster_id, C
```

Complexity: O(n × k × d × t) where n=points, k=clusters, d=dimensions, t=iterations. Converges to local optimum—multiple runs required.

Decision Tree Classifier Algorithm

Algorithm (CART): Recursively partitions data using binary splits.

Detailed Steps:
1. Splitting: For each attribute a, evaluate all possible split points s
2. Criterion: Maximize impurity reduction: $\Delta = I(parent) - \Sigma (N_v/N) \times I(child_v)$
   - Gini: $I(t) = 1 - \Sigma p(j|t)^2$
   - Entropy: $I(t) = -\Sigma p(j|t) \log p(j|t)$
3. Stopping: When node pure, or Δ < threshold, or depth reached

Example: Credit Risk Assessment

```
                Income > 50K?
               / Yes       \ No
              /             \
         Debt < 20K?      Debt < 10K?
          /   \           /   \
         / No  \ Yes     / No  \ Yes
        /       \       /       \
      HIGH     MEDIUM  HIGH     LOW
      RISK      RISK   RISK     RISK

Prediction Path: Income=40K, Debt=5K → Right branch → LOW RISK
```

Pruning: Cost-complexity pruning reduces overfitting by removing branches that provide little predictive power. The algorithm generates a maximal tree then prunes back using cross-validation.

Market Basket Analysis

Goal: Discover association rules: X → Y [support, confidence]

Example: Supermarket transactions

Itemset Lattice Visualization:

```
                {A,B,C,E} (Frequent)
               /    |    \
              /     |     \
       {A,B,C} {A,B,E} {A,C,E} {B,C,E}
        |       |       |       |
        +-------+-------+-------+-------{A,B,C,E}
              \       |       /
               \      |      /
                {A,B} {A,C} {B,E} {C,E}
                  \   |   /  /
                   \  |  /  /
                   {A} {B} {C} {E}
```


Support thresholds prune low-frequency itemsets.

Apriori Algorithm:
1. Find frequent 1-itemsets (support ≥ min_sup)
2. Generate candidate k-itemsets from frequent (k-1)-itemsets
3. Prune candidates with infrequent subsets
4. Scan database to verify support

Rule Generation:
- {Milk, Bread} → {Butter} [support=2%, conf=60%]
- Interpretation: 2% of baskets contain all three; 60% who buy milk & bread also buy butter

Lift: Measures rule significance. Lift > 1 indicates positive correlation; Lift = 1 means independence.

These comprehensive explanations provide the theoretical foundation, visual representations through text diagrams, and practical examples that demonstrate each concept's application in modern data architecture and mining contexts.

MCS 221 Data Warehouse Fundamentals and Architecture

{Block1}

{CH=1}

DATA WAREHOUSING define data warehouse. List and explain 4 chrctstcs of data warehouse(10)

Data Warehouse is used to collect and manage data from various sources, in order to provide meaningful business insights. A data warehouse is usually used for linking and analyzing heterogeneous sources of business data. Data warehouses gather data from multiple sources (including databases), with an emphasis on storing, filtering, retrieving and in particular, analyzing huge quantities of organized data. Data warehouses are used extensively in the largest and most complex businesses around the world. In demanding situations, good decision making becomes critical.

Benefits/Characterstics:

Integrated Data: One of the key characteristics of a data warehouse is that it contains integrated data. This means that the data is collected from various sources, such as transactional systems, and then cleaned, transformed, and consolidated into a single, unified view. This allows for easy access and analysis of the data, as well as the ability to track data over time.

Subject-Oriented: A data warehouse is also subject-oriented, which means that the data is organized around specific subjects, such as customers, products, or sales. This allows for easy access to the data relevant to a specific subject, as well as the ability to track the data over time.

Non-Volatile: Another characteristic of a data warehouse is that it is non-volatile. This means that the data in the warehouse is never updated or deleted, only added to. This is important because it allows for the preservation of historical data, making it possible to track trends and patterns over time.

Time-Variant: A data warehouse is also time-variant, which means that the data is stored with a time dimension. This allows for easy access to data for specific time periods, such as last quarter or last year. This makes it possible to track trends and patterns over tim

DATA WAREHOUSE DESIGN APPROACHES

Data Warehouse design approaches are very important aspect of building data warehouse. Selection of right data warehouse design could save lot of time and project cost. There are two different Data Warehouse Design Approaches Bill Inmon(Top-Down Approach) and Ralph Kimball(Bottom up Approach).

Top-down Approach: Bill Inmon's design methodology is based on a top-down approach. In the top-down approach, the data warehouse is designed first and then data mart is built on top of data warehouse.

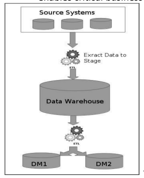
Below are the steps that are involved in top-down approach:

- Data is extracted from the various source systems. The extracts are loaded and validated in the stage area. Validation is required to make sure the extracted data is accurate and correct. You can use the ETL tools or approach to extract and push to the data warehouse.
- Data is extracted from the data warehouse in regular basis in stage area. At this step, you will apply various aggregation, summarization techniques on extracted data and loaded back to the data warehouse.
- Once the aggregation and summarization is completed, various data marts extract that data and apply the some more transformation to make the data structure as defined by the data marts.

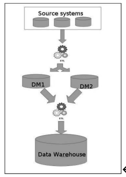
Bottom-up Approach: Ralph Kimball's data warehouse design approach is called dimensional modelling or the Kimball methodology. This methodology follows the bottom-up approach. As per this method, data marts are first created to provide the reporting and analytics capability for specific business process, later with these data marts enterprise data warehouse is created. Basically, Kimball model reverses the Inmon model i.e. Data marts are directly loaded with the data from the source systems and then ETL process is used to load in to Data Warehouse.

Below are the steps that are involved in bottom-up approach:

- The data flow in the bottom up approach starts from extraction of data from various source systems into the stage area where it is processed and loaded into the data marts that are handling specific business process.
- After data marts are refreshed the current data is once again extracted in stage area and transformations are applied to create data into the data mart structure. The data is the extracted from Data Mart to the staging area is aggregated, summarized and so on loaded into EDW and then made available for the end user for analysis and enables critical business decisions.

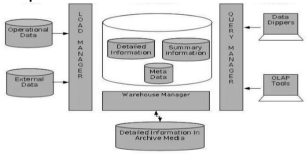


← Top down Approach



← Bottom up Approach

Components of a Data Warehouse



- Load Manager: Load Manager Component of data warehouse is responsible for collection of data from operational system and converts them into usable form for the users. This component is responsible for importing and exporting data from operational systems.
- Warehouse Manager: The warehouse manager is the center of data-warehousing system and is the data warehouse itself. It is a large, physical database that holds a vast am0unt of information from a wide variety of sources. The data within the data warehouse is organized such that it becomes easy to find, use and update frequently from its sources.
- Query Manager: Query Manager Component provides the end-users with access to the stored warehouse information through the use of specialized end-user tools. Data mining access tools have various categories such as query and reporting, on-line analytical processing (OLAP), statistics, data discovery and graphical and geographical information systems.
- End-user access tools: This is divided into the following categories, such as:
  Reporting Data
  Query Tools
  Data Dippers
  Tools for EIS
  Tools for OLAP and tools for data mining.

OLTP AND OLAP short note on OLTP (5)

- **Online Transaction Processing (OLTP)**: An OLTP system captures and maintains transaction data in a database. Each transaction involves individual database records made up of multiple fields or columns. Examples include banking and credit card activity or retail checkout scanning.
  In OLTP, the emphasis is on fast processing, because OLTP databases are read, written, and updated frequently. If a transaction fails, built-in system logic ensures data integrity.
- **Online Analytical Processing (OLAP)**: Online Analytical Processing (OLAP) applies complex queries to large amounts of historical data, aggregated from OLTP databases and other sources, for data mining, analytics, and business intelligence projects. In OLAP, the emphasis is on response time to these complex queries. Each query involves one or more columns of data aggregated from many rows.
  Examples include year-over-year financial performance or marketing lead generation trends.
  OLTP is operational, while OLAP is informational.

difference between OLTP and OLAP.

	OLTP	OLAP
Characteristics	Handles a large number of small transactions	Handles large volumes of data with complex queries
Query types	Simple standardized queries	Complex queries
Operations	Based on INSERT, UPDATE, DELETE commands	Based on SELECT commands to aggregate data for reporting
Response time	Milliseconds	Seconds, minutes, or hours depending on the amount of data to process
Purpose	Control and run essential business operations in real time	Plan, solve problems, support decisions, discover hidden insights
Productivity	Increases productivity of end users	Increases productivity of business managers, data analysts, and executives
Data view	Lists day-to-day business transactions	Multi-dimensional view of enterprise data
User examples	Customer-facing personnel, clerks, online shoppers	Knowledge workers such as data analysts, business analysts, and executives
Database design	Normalized databases for efficiency	Denormalized databases for analysis

DATA GRANULARITY short note on Data Granularity (5)

Granularity is one of the main elements in the modelling of DW data. Granularity of data refers to detail levels. Multiple levels of detail may be available depending on the requirements. At least two granular levels exist for many data warehouses. The relation between detailing and granularity is important to understand. It means greater detail of the data (less summary) when you speak of less granularity or fine granularity. Greater granularity means fewer details or gross granularity (greater summarization). The operational data is stored at the lowest level of information. The type of granularity depends on how data will be processed, and the performance expectations. For example, each year details of each month, day, hour, minute, second, and so forth are available.

TYPES OF DATA WAREHOUSES: There are three different types of traditional Data Warehouse models as listed below:

Define a data warehouse. List and explain all 3 types of data warehouse.10

(i) Enterprise Data Warehouse: An enterprise provides a central repository database for decision support throughout the enterprise. It is a central place where all business information from different sources and applications are made available. The enterprise goal is to provide a complete overview of any particular object in the data model.

(ii) Operational Data Warehouse: It assists in obtaining data straight from the database, which also helps data transaction processing. The data present in the Operational Data Store can be scrubbed, and the duplication which is present can be reviewed and fixed by examining the corresponding market rules.

(iii) Data Mart: Data Mart may be a subset of knowledge warehouse, and it supports a specific region, business unit, or business function. Data Mart focuses on storing data for a particular functional area, and it contains a subset of data saved in a memory. Data Marts help in enhancing user responses and also reduces the volume of data for data analysis.

{CH=2}

DATA WAREHOUSE ARCHITECTURE

Data warehouse architecture defines the arrangement of the data in different databases. When designing a data warehouse, there are three different types of models to consider, based on the approach of number of ***tiers*** the architecture has.

(i) Single-tier data warehouse architecture (ii) Two-tier data warehouse architecture (iii) Three-tier data warehouse architecture

1. **Single-tier data warehouse architecture:** The single-tier architecture (Figure 1) is not a frequently practiced approach. The main goal of having such architecture is to remove redundancy by minimizing the amount of data stored. Its primary disadvantage is that it doesn't have a component that separates analytical and transactional processing.
2. **Two-tier data warehouse architecture:** The two-tier architecture (Figure 2) includes a staging area for all data sources, before the data warehouse layer. By adding a staging area between the sources and the storage repository, you ensure all data loaded into the warehouse is cleansed and in the appropriate format.
3. **Three-tier data warehouse architecture:** The three-tier approach (Figure 3) is the most widely used architecture for data warehouse systems. Essentially, it consists of three tiers:
   - **The bottom tier** is the database of the warehouse, where the cleansed and transformed data is loaded.
   - **The middle tier** is the application layer giving an abstracted view of the database. It arranges the data to make it more suitable for analysis. This is done with an OLAP server, implemented using the ROLAP or MOLAP model.
   - **The top-tier** is where the user accesses and interacts with the data. It represents the front-end client layer. You can use reporting tools, query, analysis or data mining tools.

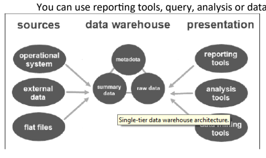


Figure 1: Single Tier Data Warehouse Architecture

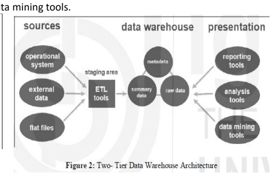


Figure 2: Two- Tier Data Warehouse Architecture

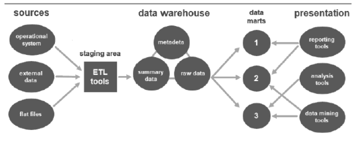


Figure 3: Three- Tier Data Warehouse Architecture

Figure 4 illustrates the complete data warehouse architecture with the three tiers:

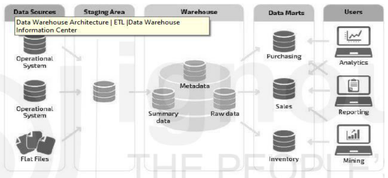


Figure 4: 3-Tiers of Data Warehouse

COMPONENTS OF DATA WAREHOUSE ARCHITECTURE

1. **Data Warehouse Database:** The central component of a DW architecture is a data warehouse database that stocks all enterprise data and makes it manageable for reporting. This means we choose the kind of database we'll use to store data in your warehouse
2. **Extraction, Transformation, and Loading Tools (ETL) (Explain detail in ch 4)**
   What is ETL? Why do you need ETL in data warehouses? Also, explain how ETL works in Data Warehousing? 10

   ETL tools are central components of enterprise data warehouse architecture. These tools help extract data from different sources, transform it into a suitable arrangement, and load it into a data warehouse.
   The ETL tool you choose will determine:
   □ The time expended in data extraction
   □ Approaches to extracting data
   □ Kind of transformations applied and the simplicity to do so
   □ Business rule definition for data validation and cleansing to improve end-product analytics
   □ Outlining information distribution from the fundamental depository to your BI applications
3. **Metadata:**

What is Metadata? What are its contents? Justify how metadata can be an important component in Data Warehousing? Also mention its types. 10

In the data warehouse architecture, metadata describes the data warehouse database and offers a framework for data. It helps in constructing, preserving, handling, and making use of the data warehouse. Metadata plays an important role for businesses and the technical teams to understand the data present in the warehouse and convert it into information. There are two types of metadata in data mining:

- **Technical Metadata** comprises information that can be used by developers and managers when executing warehouse development and administration tasks.
- **Business Metadata** comprises information that offers an easily understandable standpoint of the data stored in the warehouse.

4. **Data Warehouse Access Tools:** A data warehouse uses a database or group of databases as a foundation. Data warehouse corporations generally cannot work with databases without the use of tools unless they have database administrators available. It include
   □ Query and reporting tools
   □ Application development tools
   □ Data mining tools for data warehousing
   □ OLAP tools
5. **Data Warehouse Bus:** It defines the data flow within a data warehousing bus architecture and includes a data mart. A data mart is an access level that allows users to transfer data. It is also used for partitioning data that is produced for a particular user group.
6. **Data Warehouse Reporting Layer:** The reporting layer in the data warehouse allows the end-users to access the BI interface or BI database architecture. The purpose of the reporting layer in the data warehouse is to act as a dashboard for data visualization, create reports, and take out any required information.

LAYERS OF DATA WAREHOUSE ARCHITECTURE

The data warehouse architecture can be divided into four layers. They are:
i. Data Source Layer
ii. Data Staging Layer
iii. Data Storage Layer
iv. Data Presentation Layer

(i) Data source layer: The data source layer is the place where unique information, gathered from an assortment of inner and outside sources, resides in the social database. Following are the examples of the data source layer:
□ ***Operational Data*** — Product information, stock information, marketing information, or HR information
□ ***Social Media Data*** — Website hits, content fame, contact page completion
□ ***Outsider Data*** — Demographic information, study information, statistics information

(ii) Data Staging Layer: This layer dwells between information sources and the data warehouse. In this layer, information is separated from various inside and outer data sources. Since source data comes in various organizations, the data extraction layer will use numerous technologies and devices to extricate the necessary information. Once the extracted data has been stacked, it will be exposed to high-level quality checks. The conclusive outcome will be perfect and organized data that you will stack into your data warehouse

(iii) Data Storage Layer: This layer is the place where the data that was washed down in the arranging zone is put away as a solitary central archive. Contingent upon your business and your warehouse architecture necessities, your data storage might be a data warehouse center, data mart (data warehouse somewhat recreated for particular departments), or an Operational Data Store (ODS).

(iv) Data Presentation Layer: This is where the users communicate with the scrubbed and sorted out data. This layer of the data architecture gives users the capacity to query the data for item or service insights, break down the data to conduct theoretical business situations, and create computerized or specially appointed reports.
You may utilize an OLAP or reporting instrument with an easy to understand Graphical User Interface (GUI) to assist users with building their queries, perform analysis, or plan their reports.

DATA MARTS Short note on Data Marts (5) (again)

A data mart is a subset of a data warehouse focused on a particular line of business, department, or subject area. Data marts make specific data available to a defined group of users, which allows those users to quickly access critical insights without wasting time searching through an entire data warehouse. For example, many companies may have a data mart that aligns with a specific department in the business, such as finance, sales, or marketing.

Benefits of Using Data Mart

1. Cost-efficiency Simplified data acces Quicker access to insights Simpler data maintenance Easier and faster implementation

TYPES OF DATA MARTS

- **Dependent data marts** are partitioned segments within an enterprise data warehouse. This top-down approach begins with the storage of all business data in one central location. The newly created data marts extract a defined subset of the primary data whenever required for analysis.
- **Independent data marts** act as a standalone system that doesn't rely on a data warehouse. Analysts can extract data on a particular subject or business process from internal or external data sources, process it, and then store it in a data mart repository until the team needs it.
- **Hybrid data marts** combine data from existing data warehouses and other operational sources. This unified approach leverages the speed and user-friendly interface of a top-down approach and also offers the enterprise-level integration of the independent method.

DESIGNING THE DATA MARTS: The process for designing a data mart usually comprises the following steps:

(i)Essential Requirements Gathering: The first step is to create a robust design. Some critical processes involved in this phase include collecting the corporate and technical requirements, identifying data sources, choosing a suitable data subset, and designing the logical layout (database schema) and physical structure.

(ii) Build/Construct: The next step is to construct it. This includes creating the physical database and the logical structures. In this phase, you'll build the tables, fields, indexes, and access controls.

(iii) Populate/Data Transfer: The next step is to populate the mart, which means transferring data into it. In this phase, you can also set the frequency of data transfer, such as daily or weekly. This usually involves extracting source information, cleaning and transforming the data, and loading it into the departmental repository.

(iv) Data Access: In this step, the data loaded into the data mart is used in querying, generating reports, graphs, and publishing. The main task involved in this phase is setting up a meta-layer and translating database structures and item names into corporate expressions so that non-technical operators can easily use the data mart. If necessary, you can also set up API and interfaces to simplify data access.

(v) Manage: The last step involves management and observation, which includes:
□ Controlling ongoing user access.
□ Optimization and refinement of the target system for improved performance.
□ Addition and management of new data into the repository.
□ Configuring recovery settings and ensuring system availability in the event of failure.

{Ch=3}

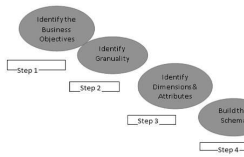
DIMENSIONAL MODELING: Dimensional Modelling (DM) is a data structure technique optimized for data storage in a Data warehouse. The purpose of dimensional modelling is to optimize the database for faster retrieval of data. A dimensional model in data warehouse is designed to read, summarize, analyze numeric information like values, balances, counts, weights, etc. in a data warehouse. The main objective of dimension modelling is to provide an easy architecture for the end user to write queries. Also it will reduce the number of relationships between the tables and dimensions, hence providing efficient query handling.

Features of Dimensional Modelling

- Easy to Understand
- Promote Data Quality
- Optimise Performance
- It reduces the number of relationships between different data elements.

Steps Involved in Dimensional Modelling

The following are the steps involved in Dimension modelling as shown in figure1.



Step 1: Identify the Business Process: Identifying the actual business process a datarehouse should cover. This could be Marketing, Sales, HR, etc. as per the data analysis needs of the organization. The selection of the Business process also depends on the quality of data available for that process. It is the most important step of the Data Modelling process

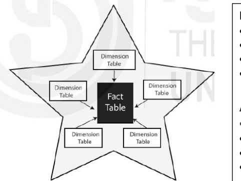
Step 2: Identifying Granularity: The Grain describes the level of detail for the business problem/solution. It is the process of identifying the lowest level of information for any table in your data warehouse. If a table contains sales data for every day, then it should be daily granularity. If a table contains total sales data for each month, then it has monthly granularity.

Step 3: Identifying Dimensions and attributes: The dimensions of the data warehouse can be understood by the entities of the database. like, items, products, date, stocks, time etc. The identification of the primary keys and the foreign keys specifications all are described here.

Step 4: Build the Schema: The database structure or arrangement of columns in a database table, decides the schema. There are various popular schemas like, star, snowflake, fact constellation schemas - summarizing, from the selection of business process to identifying each and every finest level of detail of the business transactions. Identifying the significant dimensions and attributes would help to build the schema.

STAR SCHEMA With the help of an example, explain the star schema dimensional model. 10

It represents the multidimensional model. In this model the data is organized into facts and dimensions. The star model is the underlying structure for a dimensional model. It has one broad central table (fact table) and a set of smaller tables (dimensions) arranged in a star design around the primary table. This design is logically shown in fig.



Star Schema

Figure 2: Star Schema

Features of Star Schema

- The data is in denormalized database.
- It provides quick query response
- Star schema is flexible can be changed or added easily.
- It reduces the complexity of metadata for developers and end users.

Advantages of Star Schema

- Query performance
- Load performance and administration
- Built-in referential integrity
- Easily understood

Limitations of Star Schema

- Decreased data integrity
- No Many-to-Many Relationships
- Less capable of handling diverse and complex queries

Example of Star Schema

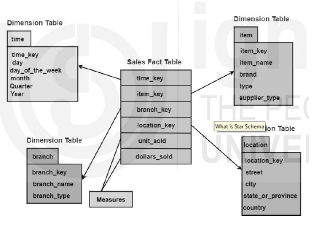


Figure 3: Example of Star Schema

Example: Suppose a star schema is composed of a fact table as shown in Figure 3, SALES, and several dimension tables connected to it for time, branch, item, and geographic locations.

The TIME table has a column for each day, month, quarter, and year. The ITEM table has columns for each item_Key, item_name, brand, type, supplier_type. The BRANCH table has columns for each branch_key, branch_name, branch_type. The LOCATION table has columns of geographic data, including street, city, state, and country.

SNOWFLAKE SCHEMA Describe the snowflake schema multidimensional modeling technique. Give an example and illustrate it. List its pros and cons. 10

A snowflake schema is a type of data modeling technique used in data warehousing to represent data in a structured way that is optimized for querying large amounts of data efficiently. In a snowflake schema, the dimension tables are normalized into multiple related tables, creating a hierarchical or "snowflake" structure.

In a snowflake schema, the fact table is still located at the center of the schema, surrounded by the dimension tables. However, each dimension table is further broken down into multiple related tables, creating a hierarchical structure that resembles a snowflake.

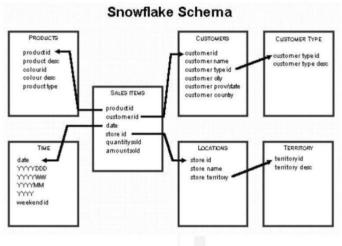


Figure 4: Snowflake Schema

Example: In the below Figure 4,

The snowflake schema is shown of a case study of customers, sales, products, location wise quantity sold, and number of items sold are calculated.

The customers, products, date, store are saved in the fact table with their respective primary keys acting in fact table as a foreign key.

In this, two aggregate functions can be applied to calculate quantity sold and amount sold. Further, some dimensions are extended to the type of customer and also store information territory wise too. Note, date has been expanded into date, month, year.

Features of Snowflake Schema

- It has normalized tables
- Occupy less disk space.
- It requires more lookup time as many tables are interconnected and extending dimensions.

Advantages of Snowflake Schema

- It provides structured data which reduces the problem of data integrity.
- It uses small disk space because data are highly structured
- Data is easy to maintain and more structured.
- Data quality is better than star schema.

Limitations of Snowflake Schema

- Snowflaking reduces space consumed by dimension tables but compared with the entire data warehouse the saving is usually insignificant.
- Avoid snowflaking or normalization of a dimension table, unless required and appropriate.
- Multiple hierarchies that can belong to the same dimension have been designed at the lowest possible detail.

Star Schema Vs Snowflake Schema

	Star Schema	Snowflake Schema
Dimension Table	The dimension tables in star schema are not normalized so they may contain redundancies	This schema has normalized dimension tables
Queries	The execution of queries is relatively faster as there are less joins needed in forming a query.	The execution of snowflake schema complex queries is slower than star schema as many joins are needed to form a query.
Performance	Star schema model has faster execution and response time	It has slow performance as compared to star schema
Storage Space	This type of schema requires more storage space as compared to snowflake due to unnormalised tables.	Snowflake schema tables are easy to maintain and save storage space due to normalized tables.
Usage	Star schema is preferred when the dimension tables have lesser rows	If the dimension table contains large number of rows, snowflake schema is preferred
Type of DW	This schema is suitable for 1:1 or 1: many relationships such as data marts.	It is used for complex relationships such as many: many in enterprise Data warehouses.
Dimension Tables	Star schema has a single table for each dimension	Snowflake schema may have more than one dimension table for each dimension.

FACT CONSTELLATION SCHEMA

There is another schema for representing a multidimensional model. This term fact constellation is like the galaxy of universe containing several stars. It is a collection of fact schemas having one or more-dimension tables in common as shown in the figure below. This logical representation is mainly used in designing complex database systems

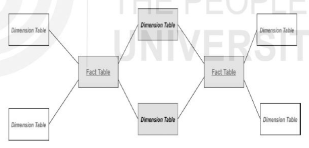


Figure 7: Fact Constellation Schema

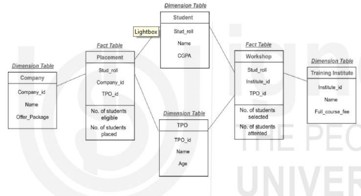


Figure 7: Fact Constellation of student and placement records

Fact tables

Placement (Stud_roll, Company_id, TPO_id) , need to calculate the number of students eligible and number of students placed.
Workshop (Stud_roll, Institute_id, TPO_id) need to find out the facts about number of students selected, number of students attended the workshop)

So, there are two fact tables namely, Placement and Workshop which are part of two different star schemas having dimension tables – Company, Student and TPO in Star schema with fact table Placement and dimension tables – Training Institute, Student and TPO in Star schema with fact table Workshop.
Both the star schema has two-dimension tables common and hence, forming a fact constellation or galaxy schema.

Advantages of Fact Constellation Schema

- Different fact tables are explicitly assigned to the dimensions.
- Provides a flexible schema for implementation

3.9.2 Limitations of Fact Constellation Schema

- Complexity of the schema involved because of several aggregations
- Fact constellation solution is hard to maintain and support

{Block-2}
{Ch=4}

ETL (Extract, Transform, Load)

What is ETL? Why do you need ETL in data warehouses? Also, explain how ETL works in Data Warehousing? 10

It is a process of data integration that encompasses three steps - ***extraction, transformation,*** and ***loading***. In a nutshell, ETL systems take large volumes of raw data from multiple sources, convert it for analysis, and load that data into your warehouse.

Why Do You Need ETL?: ETL saves you significant time on data extraction and preparation - time that you can better spend on evaluating your business. Practicing ETL is also part of a healthy data management workflow, ensuring high data quality, availability, and reliability. Each of the three major components in the ETL saves time and development effort by running just once in a dedicated data flow:

Extract: In ETL, the first link determines the strength of the chain. The extract stage determines which data sources to use, the refresh rate (velocity) of each source, and the priorities (extract order) between them — all of which heavily impact your time to insight.

Transform: After extraction, the transformation process brings clarity and order to the initial data swamp. Dates and times combine into a single format and strings parse down into their true underlying meanings. Location data convert to coordinates, zip codes, or cities/countries. The transform step also sums up, rounds, and averages

measures, and it deletes useless data and errors or discards them for later inspection. It can also mask personally identifiable information (PII) to comply with GDPR, CCPA, and other privacy requirements.

Load: In the last phase, much as in the first, ETL determines targets and refresh rates. The load phase also determines whether loading will happen incrementally, or if it will require "upsert" (updating existing data and inserting new data) for the new batches of data.

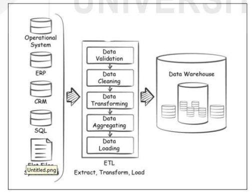


Figure 1: ETL in a Data Warehouse

ETL PROCESS: ETL collects and processes data from various sources into a single data store (a data warehouse or data lake), making it much easier to analyze. The three steps in ETL process are mentioned below:

1. **Data Extraction:** Data extraction involves the following four steps:
   - ***Identify the data to extract:*** The first step of data extraction is to identify the data sources. These sources might be from relational SQL databases like MySQL or non-relational NoSQL databases like MongoDB or Cassandra
   - ***Estimate how large the data extraction is:*** The size of the data extraction matters. A larger quantity of data will require a different ETL strategy.
   - ***Choose the extraction method***:
2. **Data Transformation**: What is data transformation? List and explain all the data transformation steps (10)
   data transformation that occurs in a staging area (after extraction) is "multistage data transformation". In ELT, data transformation that happens after loading data into the data warehouse is "in-warehouse data transformation". Some of the following data transformations are:
   - ***Deduplication (normalizing):*** Identifies and removes duplicate information.
   - ***Key restructuring***: Draws key connections from one table to another.
   - ***Cleansing:*** Involves deleting old, incomplete, and duplicate data to maximize data accuracy - perhaps through parsing to remove syntax errors, typos, and fragments of records.
   - ***Format revision***: Converts formats in different datasets - like date/time, male/female, and units of measurement - into one consistent format.
   - ***Derivation***: Creates transformation rules that apply to the data. For example, maybe you need to subtract certain costs or tax liabilities from business revenue figures before analyzing them.
   - ***Aggregation:*** Gathers and searches data so you can present it in a summarized report format.
   - ***Integration:*** Reconciles diverse names/values that apply to the same data elements across the data warehouse so that each element has a standard name and definition.
   - ***Filtering:*** Selects specific columns, rows, and fields within a dataset.
   - ***Splitting:*** Splits one column into more than one column.
   - ***Joining:*** Links data from two or more sources, such as adding spend information across multiple SaaS platforms.

- ***Summarization:*** Creates different business metrics by calculating value totals. For example, you might add up all the sales made by a specific salesperson to create total sales metrics for specific periods.
- ***Validation***: Sets up automated rules to follow in different circumstances. For instance, if the first five fields in a row are NULL, then you can flag the row for investigation or prevent it from being processed with the rest of the information.

3. **Data Loading:** Data loading is the process of loading the extracted information into your target data repository. Loading is an ongoing process that could happen through "full loading" (the first time you load data into the warehouse) or "incremental loading" (as you update the data warehouse with new information). Because incremental loads are the most complex.

WORKING OF ETL

1. **Parsing/Cleansing:** Data generated by applications may be in various formats like JSON, XML, or CSV. The parsing stage maps data into a table format with headers, columns, and rows, and then extracts specified fields.
2. **Data enrichment:** Preparing data for analytics usually requires certain data enrichment steps, including injecting expert knowledge, resolving discrepancies, and correcting bugs.
3. **Setting velocity:** "Velocity" refers to the frequency of data loading, i.e. inserting new data and updating existing data.
4. **Data validation:** In some cases, data is empty, corrupted, or missing crucial elements. During data validation, ETL finds these occurrences and determines whether to stop the entire

ELT (Extract/load/transform)

Extract/load/transform (ELT) is the process of extracting data from one or multiple sources and loading it into a target data warehouse. Instead of transforming the data before it's written, ELT takes advantage of the target system to do the data transformation. This approach requires fewer remote sources than other techniques because it needs only raw and unprepared data.

- Extract - This step works similarly in both ETL and ELT data management approaches. Raw streams of data from virtual infrastructure, software, and applications are ingested either in their entirety or according to predefined rules.
- Load – The ELT differs here with the ETL. Rather than deliver this mass of raw data and load it to an interim processing server for transformation, ELT delivers it directly to the target storage location. This shortens the cycle between extraction and delivery.
- Transform - The database or data warehouse sorts and normalizes the data, keeping part or all of it on hand and accessible for customized reporting. The overhead for storing this much data is higher, but it offers more opportunities to mine it for relevant business intelligence in near real-time.

Need: The ELT process improves data conversion and manipulation capabilities due to parallel load and data transformation functionality.
The ELT process saves you steps and time. Data is first loaded into the target ecosystem, such as a data warehouse, and then transformed. Authorized users can securely access the data without returning it to source systems. No downloading is necessary for it.

ETL Vs ELT

The primary differences between ETL and ELT are how much data is retained in data warehouses and where data is transformed. With ETL, the transformation of data is done before it is loaded into a data warehouse. This enables analysts and business users to get the data they need faster, without building complex transformations or persistent tables in their business intelligence tools. Using the ELT approach, data is loaded into the warehouse as is, with no transformation before loading into it.

{Ch=5}

ONLINE ANALYTICAL PROCESSING(OLAP)

Online Analytical Processing (OLAP) is the technology to analyze and process data from multiple sources at the same time. It accesses the multiple databases at the same time. It helps the data analysts to collect data from different perspective for developing effective business strategies. OLAP is a software used to perform high-speed, multivariate analysis of large amounts of data in data warehouses, data markets, or other unified and centralized data warehouses. The data is broken down for display, monitoring or analysis

Multidimensional Operations List and describe all the four types of OLAP operations. 10

Typically, there are four types of OLAP operations are:
i. Roll-up
ii. Drill-down
iii. Slice and Dice
iv. Pivot (rotate)

1. ***Roll-up:*** It is an OLAP operation to move from low level of concept hierarchy to higher level known as aggregation. It can be performed by reducing the dimensions as well.
2. ***Drill-down:*** In this process of scaling down from higher level to the lower level the data is broken into smaller parts.
3. ***Slice:*** It is another OLAP operation to fetch the data. In this the query on one dimension is triggered in the database and a new sub cube is created
4. ***Dice:*** This OLAP Dice operation as shown in figure 6 is just like the Projection relational query you have read in RDBMS. In this technique, you select two or more dimensions that results in the creation of a sub cube
5. ***Pivot:*** This OLAP operation fixes one attribute as a Pivot and rotate the cube to fetch the results, just like an inverted spreadsheet, giving it a different perspective.

OLAP ARCHITECTURE: MOLAP, ROLAP, HOLAP AND DOLAP

There are various types of OLAP architecture: ROLAP, MOLAP, HOLAP and DOLAP and others

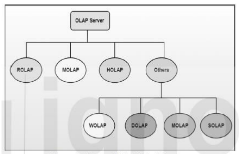


Figure 9: Types of OLAP Architecture

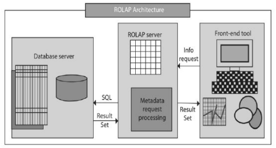


Figure 10 : ROLAP Architechture

ROLAP Architecture

ROLAP implies Relational OLAP, an application based on relational DBMSs. It performs dynamic multidimensional analysis of data stored in a relational database. The architecture is like three-tier. It has three components viz. front end (User Interface), ROLAP server (Metadata request processing engine) and the back end (Database Server).
In this three-tier architecture the user submits the request and ROLAP engine converts the request into SQL and submits to the backend database. After the processing of request by the engine, it presents the resulting data into multidimensional format to make the task easier for the client to view it. The architecture has three components:

- Database server. ● ROLAP server. ● Front-end tool

MOLAP Architecture

Explain the following OLAP architectures and draw their architectural diagram: 10
(i) Multidimensional Online Analytical Processing (MOLAP)
(ii) Hybrid Online Analytical Processing (HOLAP)

MOLAP stands for Multidimensional Online Analytical Processing. It processes the data using the multidimensional cube using various combinations. Since, the data is stored in multidimensional structure the MOLAP engine uses the pre-computed or pre-stored information. MOLAP engine processes pre-compiled information. It has dynamic abilities to perform aggregation of concept hierarchy. MOLAP is very useful in time-series data analysis and economic evaluation. The architecture has three components:
● Database server ● MOLAP server ● Front-End tool

Characteristics of MOLAP

- It is a user-friendly architecture, easy to use.
- The OLAP operations slice and dice speeds up the data retrieval.
- It has small pre-computed hypercubes.

HOLAP Architecture

It defines Hybrid Online Analytical Processing. It is the hybrid of ROLAP and MOLAP technologies. It connects both the dimensions together in one architecture. It stores the intermediate or part of the data in ROLAP and MOLAP. Depending on the query request it accesses the databases. It stores the relational tables in ROLAP structure, and the data requires multidimensional views, stored and processed using MOLAP architecture as shown in figure 12.
It has the following components:
● Database server. ● ROLAP and MOLAP server. ● Front-end tool

Characteristics of HOLAP are:

- Flexible handling of data.
- Faster aggregation of data.
- HOLAP can drill down the hierarchy of data and can access to relational database for any relevant and stored information in it.

DOLAP Architecture

Desktop Online Analytical Processing (DOLAP) architecture is most suitable for local multidimensional analysis. It is like a miniature of multidimensional database or it's like a sub cube or any business data cube. The components are:

- Database Server. ● DOLAP server. ● Front-end

characteristics of DOLAP are:

- The three-tier architecture is designed for low-end, standalone user like a small shop owner in the locality.
- The data cube is locally stored in the system so, retrieval of results is faster.
- No load on the backend or at the server end.
- DOLAP is relatively cheaper to deploy (NO DIAGRAM OF DOLAP)

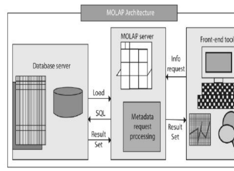


Figure 11 : MOLAP Architechture

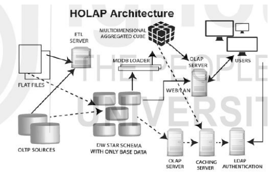


Figure 12 : HOLAP Architecture

{CH=6}

DATA LAKES Define a Data Lake. Explain the step-by-step process of creating a data lake. 10

A data lake is a central location that holds a large amount of data in its native, raw format. data lake uses a flat architecture and object storage to store the data. It can store structured, semi-structured, or unstructured data, which means data can be kept in a more flexible format for future use. When storing data, a data lake associates it with identifiers and metadata tags for faster retrieval.

Step by step process me koi dhng ki language ni mili.... Net pr smjh kr apni language me likh denge

Complex data modelling

What is complex data modelling? Where is this used? Explain "Anchor" complex data model. (10)

Complex data travels through data processing tools, ETLs, ERP software, data lakes and other paths. Complex data has its intrinsic traits like its own syntax, schema, technology, terminology and type. This diversity of traits complicates the work of data modelers. Various models like statistical, polyglot, no payload, multi-level, etc. exist to store, process and analyze complex data. Some standard models are also available like Anchor models and Data Vault Models. Both of these models provide advantages like Scalability, Temporal data handling and are resilient to structural and content-based changes.

Complex data models

1. Anchor Model: Anchor modelling has four elements to it:
   - ***Anchors:*** The anchors are used to model entities and events
   - ***Attributes:*** Attributes are used to model properties of anchors.
   - ***Ties:*** Ties are used to model the relationships between anchors.
   - ***Knots:*** Knots are used in the modelling of shared properties, such as states. Attributes and ties can be versioned (termed as historization), when changes in the information the model need to be retained.
     The different graphical symbols, representing the elements of the model, are similar to those used in entity-relationship modelling with a few extensions. An outline on a tie or attribute depicts versioning of changes.

2. Data Vault Model: The model also allows coping with issues such as auditing, data traceability, data loading performance and resilience to change. Data traceability implies that every row of data in a data vault must be associated with a record sourc e and date-of-load attributes. This information allows an auditor to trace values to their respective sources
   - ***Hubs:*** Hubs are comprised of a list of distinct business keys with low affinity to change and contain a surrogate key per Hub item. The Hub also contains metadata designating the source of the business key. Satellite tables;
   - ***Links:*** Link tables are models providing relations between business keys and are fundamentally many-to-many join tables, containing some metadata.
     Links can connect to other links, to handle granularity variations. Using link references within another link is bad modelling practice, as it creates dependencies between links making parallel loading of links more challenging.
   - ***Satellites:*** The hubs and link's temporal attributes and descriptive attributes stored in separate constructs called satellite tables or simply satellites. The satellites also comprise metadata connecting them to their parent link or hub, metadata designating the relationship source and attributes, also including a time-series with the attribute's start and end dates.

CLOUD DATA WAREHOUSING short note on Cloud Data Warehousing (5)

What is Hadoop?

Hadoop is an open-source java-oriented framework comprising two fundamental components: i) Storage and ii) Processing and was developed, especially, to deal with massive volumes of data in a distributed computing ecosystem. The storage component of Hadoop is called the Hadoop Distributed File System (HDFS) while the processing component is termed as MapReduce.
modules:

1. The Hadoop Distributed File System: The HDFS allows data storage with easy accessibility. The data storage resides in an environment guaranteeing *highly reliability* and *highly availability*.
2. MapReduce: MapReduce is principally a combination of two processes. Mapping and Reduction. Data is read from the data sources and converted to a format conducive to data analysts. This is the mapping process. The data is then subjected to mathematical operations like aggregations, grouping, encoding rates, etc. also called as reductions and hence the terminology - *reduce*. In the Hadoop framework, it is the MapReduce programming model implementation for processing data on a large scale also termed as *data processing at scale*.
3. Hadoop Common: Hadoop common is a collection of libraries and tools required by other Hadoop modules.
4. YARN: YARN is a platform that orchestrates and manages clustered computing resources and uses them for scheduling user-applications.
5. Hadoop Ozone: Ozone is an object store (a data storage architecture that manages data as objects, through a combination of a globally unique identifier, metadata and the data itself

Data warehouse automation tools:
A DWA tool provides a seamless experience, free from the hassles of coding, for integration and transition of diverse data from its source to a DW and other related components. The tool automates deployment of ETL scripts, batch-processing of data and demonstrates various functions like:

- Processes for high performance ETL and reliable ELT based integration of data
- Modelling of data at source
- Normalized, De-normalized and data structures with multiple dimensions
- Seamless integration with various data sources

{Block-3}
{Ch=7}

DATA MINING
What is data mining? List out the key differences between data warehousing and data mining. Mention any four applications of data mining. 10

Data mining is the process by which the patterns can be found by filtering the data based on a condition of data sorting in order to find something important. At a micro- scale, mining can be defined as an activity that involves gathering data in one place in a structural way. For example, clubbing an Excel Spreadsheet or creating a summarization of the main points of some text.

- Data mining finds valuable information hidden in large volumes of data.
- Data mining is the analysis of data and the use of software techniques for finding patterns and regularities in sets of data.
- The computer is responsible for finding the patterns by identifying the underlying rules and features in the data.

The purpose of mining the data can be multi-fold which includes:
- Predicting various outcomes;
- Modelling target audience;
- Collecting the information about the product.

Data Mining Applications: Here is the list of areas where data mining is widely used –
- Financial Data Analysis
- Retail Industry
- Telecommunication Industry
- Biological Data Analysis
- Other Scientific Applications
- Intrusion Detection

Data Mining Vs Data Warehouse

Data Mining	Data Warehouse
Data mining is the process of analyzing unknown patterns of data.	A data warehouse is database system which is designed for analytical instead of transactional work.
Data mining is a method of comparing large amounts of data to finding right patterns.	Data warehousing is a method of centralizing data from different sources into one common repository.
Data mining is usually done by business users with the assistance of engineers.	Data warehousing is a process which needs to occur before any data mining can take place.
Data mining is the considered as a process of extracting data from large data sets.	On the other hand, Data warehousing is the process of pooling all relevant data together.
One of the most important benefits of data mining techniques is the detection and identification of errors in the system.	One of the pros of Data Warehouse is its ability to update consistently. That's why it is ideal for the business owner who wants the best and latest features.

DATA MINING TOOLS

Orange Data Mining

SAS Data Mining: SAS stands for Statistical Analysis System. It is a product of the SAS Institute created for analytics and data management.

Rattle: Rattle is a data mining tool based on GUI. It uses the R programming language. Rattle combines the power of R with significant data mining features.

Rapid Miner: Rapid Miner is one of the most popular tool for performing predictions. It is written in JAVA programming language. It offers an integrated environment for text mining, deep learning, machine learning and predictive analysis.

KNIME: KNIME is open source software for creating data science applications and services. This Data mining tool helps you to understand data and to design data science workflows.

{CH=8}

Data Preprocessing Techniques:

- Data cleaning may be used to prevent noise and to correct incoherence of data.
- Data aggregation combines data from multiple channels into a cohesive data store, for example a data center.
- Data transformations may be applied, such as standardization.
- Data reduction will, for example, reduce data size by the compilation, elimination of duplicate components or clustering; it can operate together.

DATA CLEANING
Define "data cleaning" which is a data preprocessing technique. In this context, explain the concept of Noisy data cleaning along with some suitable examples. 10

The other problem is that findings in the modern world are inconsistent, noisy and inexact. Software – cleaning (or data cleaning) procedures are intended to complete missing values, smooth out sounds as outliers are identified and data errors are corrected.

1. **Missing Values**: Some tuples have no significance mentioned for a range of attributes, such as consumer income. We're going to fill up the vacant values.

2. **Noisy Data**: Noisy is a measured random error or variance in the variable. Noise is minimized by techniques of data smoothing.
3. **Data cleaning as a process:**
   Discrepancy Detection: The first step in data cleaning is to identify discrepancies. It considers the meta data awareness and examines the following rules in order to identify the distinction.
   Data Transformation: This is the second step of the data cleaning process. After identifying inconsistencies, we have to define (sequence of) transformations and execute to correct them.
   Mostly chize chodi h is chapter ki

{Ch=9}

ASSOCIATION RULE MINING

Short note on Association Rule Generation (5)

What is Association Rule Mining? With reference to this, explain the concepts given below: 10
(i) Support Count (ii) Frequent Item set (iii) Association Rule (iv) Rule Evaluation metrics

- Association rule mining is a popular and well researched method for discovering interesting relations between variables in large databases.
- It is intended to identify strong rules discovered in Databases using different measures of interestingness.
- Based on the concept of strong rules, Rakesh Aggarwal et al. introduced association rules.

Rule Evaluation Metrics –

Support: It is one of the measures of interestingness. This tells about the usefulness and certainty of rules.
5% Support means total 5% of transactions in the database follow the rule.
Support(A -> B) = Support_count(A **U** B)

Support count(X): Number of transactions in which X appears. If X is A **union** B then it is the number of transactions in which A and B both are present.
For example, if a dataset contains 100 transactions and the item set {milk, bread} appears in 20 of those transactions, the support count for {milk, bread} is 20.

Confidence(c) –
It is the ratio of the no of transactions that includes all items in {B} as well as the no of transactions that includes all items in {A} to the no of transactions that includes all items in {A}.

Frequent item sets: A frequent item set is a set of items that occur together frequently in a dataset. The frequency of an item set is measured by the support count, which is the number of transactions or records in the dataset that contain the item set. Frequent item sets and association rules can be used for a variety of tasks such as market basket analysis, cross-selling and recommendation systems.

Apriori Algorithm
Short note on Apriori Algorithm (5)

Apriori algorithm is given by R. Agrawal and R. Srikant in 1994 for finding frequent itemsets in a dataset for boolean association rule. Name of the algorithm is Apriori because it uses prior knowledge of frequent itemset properties. We apply an iterative approach or level-wise search where k-frequent itemsets are used to find k+1 itemsets.

Apriori algorithm refers to the algorithm which is used to calculate the association rules between objects. It means how two or more objects are related to one another. In other words, we can say that the apriori algorithm is an association rule leaning that analyzes that people who bought product A also bought product B.
Apriori algorithm helps the customers to buy their products with ease and increases the sales performance of the particular store.

Apriori algorithm refers to an algorithm that is used in mining frequent products sets and relevant association rules. Generally, the apriori algorithm operates on a database containing a huge number of transactions. For example, the items customers but at a Big Bazar.

Components of Apriori algorithm: The given three components comprise the apriori algorithm.

1. Support 2. Confidence 3. Lift

APPROACHES FOR MINING MULTILEVEL ASSOCIATION RULES

Uniform Minimum Support:
- The same minimum support threshold is used when mining at each level of abstraction.
- When a uniform minimum support threshold is used, the search procedure is simplified.
- The method is also simple in that users are required to specify only one minimum support threshold.

Reduced Minimum Support
- Each level of abstraction has its own minimum support threshold.
- The deeper the level of abstraction, the smaller the corresponding threshold is.
- For example, the minimum support thresholds for levels 1 and 2 are 5% and 3%, respectively. In this way, computer, laptop computer, and desktop computer‖ are all considered frequent.

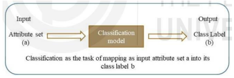
Group-Based Minimum Support:
- Because users or experts often have insight as to which groups are more important than others, it is sometimes more desirable to set up user-specific, item, or group based minimal support thresholds when mining multilevel rules.

{Block 4}
{Ch=10}

CLASSIFICATION

Define classification. With the help of an example for each, explain the following classification models: 10
(i) Descriptive modelling
(ii) Predictive modelling

A classification problem requires that examples be classified into one of two or more classes. A classification can include reliable or discrete input variables. A two-class dilemma is also regarded as a challenge between two classes or binary classification. A multi-class grouping problem is often called a two-group problem.
Classification is the activity of learning a target function "c," which maps each attribute to one of the class labels "b"



Classification as the task of mapping as input attribute set a into its class label b

The attribute set 'a' can contain a number of attributes and can contain binary, categorical and continuous attributes. The class 'b' mark must have a distinct, i.e binary or categorical attribute (nominal or ordinal)

Applications of Classification Models:
- Spam detection e-mails based on the header and content of the document.
- Classification of galaxies according to their types.
- Classification of students according to their qualifications.
- Patients are classified according to their medical history.
- Classification can be used for the approval of credit.

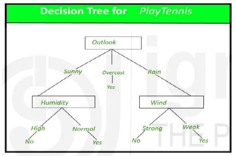
Classification Models:

Descriptive Modelling: It is a model of classification used to summarize data.
Predictive Modelling: It is a classification model used to predict the class mark of unknown data.

Decision Tree:

What is a Decision Tree? How is it useful in classification? With the help of an example, explain the process of construction of a decision tree and its representation. 10

Decision tree is the most effective and common prediction and classification method. A decision tree is a flux chart-like tree structure, where each internal node indicates an attribute test, each branch is the product of the test, and each leaf node has a class name.



Construction of Decision Tree:

A tree can be "learned" by dividing the source into subsets based on a value test attribute. This process is replicated in a recursive way, called recursive partitioning on each derived subset. The recursion is done when the subset at a node has all the same value as the target variable or when splitting doesn't add any more value to the forecasts. The construction of a decision tree classification involves no domain knowledge or parameter setting, so it is ideal for exploration of explorative knowledge. High dimensional data can be handled by decision trees. The decision tree classifier has excellent accuracy in general. Induction of decision tree is a standard inductive way of learning classification information.

Decision Tree Representation:

Decision trees identify instances by sorting them from the root to a leaf node that provides the instance classification. An instance is classified by starting from the tree root node, evaluating the specified attribute by this node, then moving down the tree branch corresponding to the attribute's value, as shown in the figure above. This process is repeated for the subtree that is rooted in the new node.

Baye's Theorem
Short note (5)

The theorem of Bayes named after British mathematician Thomas Bayes from the 18th century is a mathematical method to determine the probability of a condition. Conditional probability is the probability of an outcome, based on a prior result. The theorem of Bayes offers a way to revisit current predictions or hypotheses (update probabilities) with new or additional proof. The Bayes theorem can be used in finance to assess the danger of lending capital to potential lenders.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

{Ch=11}

CLUSTERING
Short note on Clustering and its Methods (5)

A cluster is a grouping of comparable items that belongs to the same category.
A group of comparable data objects is classed as a cluster in clustering. A collection of data is referred to as a group. The cluster analysis divides data into groupings based on how closely they resemble one another. Clustering is an unsupervised Machine Learning-based Algorithm that divides a set of data points into clusters so that the objects are all part of the same group. Using clustering, data can be subdivided into smaller, more manageable groups. Data from each of these subgroups are grouped together into a single cluster.

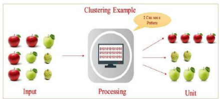


Fig: Example of Clustering

Applications of cluster analysis in Data Mining:
- Clustering analysis is widely utilized in a variety of fields, including data analysis, market research, pattern identification, and image processing.
- It aids marketers in identifying distinct customer segments based on their customers' purchasing habits. They are able to classify their clients.
- It aids data discovery by assigning documents on the internet.
- For example, credit card fraud detection relies on clustering.

Categorization of Major Clustering Methods:

1. Partitioning Method: There are a few requirements that must be met with this Partitioning Clustering Method, and they are as follows:
   Only one group should have a single objective at any given time.
   There should be no organization that does not have a single goal.
2. Hierarchical Method: This method decomposes a set of data items into a hierarchy. Depending on how the hierarchical breakdown is generated, we can put hierarchical approaches into different categories.
3. Density-based Method
4. Grid-Based Method
5. Model-Based Method
6. Constraint-based Method

{Ch=12}

Text Mining

Define text mining. Mention any three applications of it. What are the various techniques used to analyse the web-usage patterns? 10

What is Text Mining? Where is it used? Explain any two text mining techniques with the help of a suitable example for each. 10

Text mining is a component of data mining that deals specifically with unstructured text data. Text mining can be used as a preprocessing step for data mining or as a standalone process for specific tasks.
By using text mining, the unstructured text data can be transformed into structured data that can be used for data mining tasks such as classification, clustering, and association rule mining. This allows organizations to gain insights from a wide range of data sources, such as customer feedback, social media posts, and news articles.

Text mining is widely used in various fields, such as natural language processing, information retrieval, and social media analysis. It has become an essential tool for organizations to extract insights from unstructured text data and make data-driven decisions.

Procedures for Analysing Text Mining
- **Text Summarization:** To extract its partial content and reflect its whole content automatically.
- **Text Categorization:** To assign a category to the text among categories predefined by users.
- **Text Clustering:** To segment texts into several clusters, depending on the substantial relevance.

Application of Text Mining:

Digital Library Academic and Research Field Life Science Social-Media Business Intelligence

Text Mining Techniques

1. **Information Retrieval:** In the process of Information retrieval, we try to process the available documents and the text data into a structured form so, that we can apply different pattern recognition and analytical processes. It is a process of extracting relevant and associated patterns according to a given set of words or text documents.
2. **Information Extraction:** It is a process of extracting meaningful words from documents.
   - Feature Extraction – In this process, we try to develop some new features from existing ones. This objective can be achieved by parsing an existing feature or combining two or more features based on some mathematical operation.
   - Feature Selection – In this process, we try to reduce the dimensionality of the dataset which is generally a common issue while dealing with the text data by selecting a subset of features from the whole dataset.
3. **Natural Language Processing:** Natural Language Processing includes tasks that are accomplished by using Machine Learning and Deep Learning methodologies. It concerns the automatic processing and analysis of unstructured text information

WEB MINING

Web mining as the name suggests that it involves the mining of web data. The extraction of information from websites uses data mining techniques. It is an application based on data mining techniques. The parameters generally to be mined in web pages are hyperlinks, text or content of web pages, linked user activity between web pages of the same website or among different websites.
Web Mining performs various tasks such as:
1) Generating patterns existing in some websites, like customer buying behavior or navigation of web sites.
2) The web mining helps to retrieve faster results of the queries or the search text posted on the search engines like Google, Yahoo etc.
3) The ability to classify web documents according to the search performed on the ecommerce websites helps to increase businesses and transactions

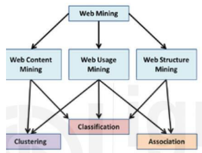


Figure 5: Three types of Web Mining (source: internet)

WEB CONTENT MINING

Web content mining is used for taking out information, data from web pages. It extracts the information and knowledge from the content of web pages. In web content mining each page is considered as an individual document
- The usefulness of web content mining is that it helps in distinguish topics or heading content of the web pages.
- Web content mining scans or reads the images or text and group of web pages. The input queries posted by the users and display the result of search engines. For example, if a user wants to search some information about some hotel in a particular region, the list will be displayed.

WEB STRUCTURE MINING
short note on Web structure mining (5)

Web structure mining helps the web pages to extract knowledge. This task is not easy as the challenge in accessing the unstructured web data link is huge requires lot of computational costs. It is used to find out the link structural parameters of hyperlink. It helps to find out the network link or if the hyperlink is directly associated with the data. The social network analysis in web structure mining treats web as a directed graph, in which the webpages are as vertices associated with the hyperlinks.

MINING MULTIMEDIA DATA ON THE WEB

In the context of mining multimedia data on the web, explain the following terms: 10
Page Rank (ii) Hits (iii) Page Layout Analysis (iv) Vision Page Segmentation

- *PageRank:* This measure is used to count the number of pages the webpage is connected to other websites. It gives the importance of the webpage. The Google search engine uses the algorithm PageRank and rank the web page very significant if is frequently connected with the other webpages on the social network.
- *HITS:* This measure is used to rate the webpage. It is developed by Jon Kleinberg. It uses hubs and authorities to be determined from a web page. Hubs and Authorities defines a recursive relationship between web pages. This algorithm helps in web link structure and speeds up the search operation of a web page. Given a query to a Search Engine, the set of highly relevant web pages are called Roots. They are potential Authorities.
  Pages that are not very relevant but point to pages in the Root are called Hubs. Thus, an Authority is a page that many hubs link to whereas a Hub is a page that links to many authorities.
- *Page Layout analysis:* It extracts and maintains the page-to-block, block-to-page relationships from link structure of web pages.
- *Vision page segmentation (VIPS) algorithm*: It first extracts all the suitable blocks from the HTML Document Object Model (DOM) tree, and then it finds the separators between these blocks. Here separators denote the horizontal or vertical lines in a Web page that visually cross with no blocks. Based on these separators, the semantic tree of the Web page is constructed.